RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
****
UNIVERSITÉ DE DSCHANG
****
ÉCOLE DOCTORALE



REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
****
UNIVERSITY OF DSCHANG
****
POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCE AND TECHNOLOGY

UNITÉ DE RECHERCHE EN INFORMATIQUE FONDAMENTALE, INGÉNIERIE ET APPLICATION (URIFIA)

THÈME :

EXTENSION DES PRÉDICATS TEMPORELS, UTILISATION DES MATRICES ET PARALLÉLISATION POUR L'EXTRACTION DES MODÈLES MULTI-TEMPORELS DANS LES BASES DE DONNÉES TEMPORELLES

Mémoire soutenu publiquement en vue de l'obtention du diplôme de Master en Informatique

Option : *Informatique Fondamentale*

Spécialité : *Intelligence Artificielle*

Présenté par :

NGWANO TUEAM Loic Dimitri

Matricule : CM-UDS-19SCI1027

Licencié en informatique fondamentale

Sous la direction de :

TAYOU DJAMEGNI Clementin

Professeur, Université de Dschang

Année 2024

DÉCLARATION D'AUTHENTICITÉ

Je soussigné, Ngwano Tueam Loic Dimitri, étudiant en Master 2, immatriculé sous le numéro CM-UDS-19SCI1027, option Informatique Fondamentale, spécialité Intelligence Artificielle au Département de Mathématiques et d'Informatique de la Faculté des Sciences de l'Université de Dschang, atteste que ce mémoire intitulé "**Extension des prédicats temporels, utilisation des matrices et parallélisation pour l'extraction des modèles multi-temporels dans les bases de données temporelles**", effectué à l'Unité de Recherche en Informatique Fondamentale, Ingénierie et Applications (URIFIA) sous la supervision du Pr Tayou Djamegni Clementin, professeur à l'Université de Dschang (Faculté des Sciences), est le fruit de mes propres recherches. Cette dissertation est authentique et n'a jamais été présentée pour l'obtention d'un diplôme ou d'un grade universitaire.

Auteur
NGWANO TUEAM Loic Dimitri

Superviseur
TAYOU DJAMEGNI Clementin
(Professeur, Université de Dschang)

Chef de Département
Pr. NKENLIFACK Marcellin
(Maître de Conférences, Université de Dschang)

DÉDICACES

À mon père TUEAM JEAN SERAPHIN *et à ma mère* KUISSU ALICE SANDRINE NADIE.

REMERCIEMENTS

Je tiens à exprimer mes remerciements les plus sincères :

— À **Dieu Tout-Puissant** sans qui rien n'aurait été possible ;

— Au **Professeur TAYOU DJAMEGNI Clémentin** pour la confiance qu'il m'a témoigné en acceptant la direction scientifique de mes travaux. Je lui suis reconnaissant de m'avoir fait bénéficier tout au long de ce travail de sa rigueur intellectuelle, de sa disponibilité, de sa bienveillance et de son efficacité. Au-delà de sa casquette d'encadreur, il est un père pour nous ;

— À tous les **enseignants du Département de Mathématiques-Informatique** pour leur aide et leurs encouragements constants ;

— À tous les **membres du jury**, pour l'immense honneur que vous m'accordez en appréciant ce travail ;

— Au **Docteur ZEKENG NDADJI Milliam Maxime**, mon mentor depuis le début de mon parcours universitaire. Merci pour tout, Docteur ;

— À **Monsieur SANWOUO Brell**, mon mentor depuis le début de mon parcours au cycle Master. Merci pour tout, Monsieur Brell ;

— À mes aînés académiques (**Daniela, Baudouin, Ramsès, Duclair, Arlèon, Brondon, Ariel, Bernoulli, Léna, Bénédicte, PANGOP**) ;

— À **TUEAM Manuela**, pour ton soutien inébranlable et tes bénédictions infinies. Manuela, tu as été une lumière dans les moments les plus sombres, une source de réconfort et de motivation inestimable. Ton aide précieuse et ton amour inconditionnel ont été les piliers sur lesquels j'ai pu m'appuyer. Merci du fond du cœur pour tout ce que tu as fait pour moi ;

— À maman **KEMBOU Diana**, pour le soutien et ses bénédictions ;

— À ma tante, **Merline Flore**, pour l'immense soutien mental ;

— À mes frères et sœurs (**Manuela, Audrez, Danielle, Imelda, Ange, Berol**) ;

— À **NGANKEU Dorcas**, celle qui a toujours été présente dans ma vie, pour tout son amour, ses encouragements et ses prières ;

— À **Sylvanie**, une amie précieuse ;

— À mes alliés, frères et amis (**Rivaldes, Ibrahim, Donald, Franck**) ;

— À mes amis, qui sont toujours là et avec qui je partage la même idéologie ;

— À tous mes camarades de promotion de **Master 2 Recherche** pour les discussions constructives que nous avons partagées ;
— À tous ceux qui m'ont soutenu et dont j'ai omis de mentionner les noms ;
— En mémoire de ma chère amie et camarade de classe, **DIFFOUO Charlotte**, ton souvenir continue de m'inspirer.

TABLE DES MATIÈRES

Dédicaces . . . ii
Remerciements . . . iii
Table des Matières . . . v
Liste des Symboles . . . viii
Liste des Acronymes . . . ix
Liste des tableaux . . . x
Table des figures . . . xi
Résumé . . . xii
Abstract . . . xiv

Introduction générale **1**
Contexte du travail . . . 1
Problématique étudiée . . . 2
Objectif . . . 2
Contribution . . . 3
Plan de travail . . . 3

Chapitre I ► **Généralités sur la fouille de données et sur la parallélisation** **5**
I.1 - Introduction . . . 5
I.2 - Généralités sur la fouille de données . . . 6
I.2.1 - Définition et historique . . . 6
I.2.2 - Les techniques et les algorithmes de fouille de données . . . 7
I.2.2.1 - La classification . . . 7
I.2.2.2 - La régression . . . 10
I.2.2.3 - Le clustering . . . 11
I.2.2.4 - La règles d'association . . . 12
I.2.2.5 - Analyse de séries temporelles . . . 13
I.2.2.6 - Les motifs séquentiels . . . 14
I.2.3 - Les applications de la fouille de données . . . 17
I.2.3.1 - Secteurs d'application . . . 17
I.2.3.2 - Cas d'usage concrets . . . 17

I.2.4 - Les défis et les limitations de la fouille de données 18
I.2.4.1 - Qualité des données 18
I.2.4.2 - Problèmes éthiques et de confidentialité 19
I.2.4.3 - Complexité computationnelle 19
I.3 - Généralités sur la parallélisation 19
I.3.1 - L'évolution des technologies et le besoin de parallélisation . . 19
I.3.2 - Classifications des architectures parallèles 21
I.3.2.1 - Classification de Flynn 21
I.3.2.2 - Classification de Raina 22
I.3.2.3 - Classification suivant le réseau d'interconnexion . . . 24
I.3.2.4 - Classification suivant la granularité de calcul 25
I.3.3 - Modèles de programmation parallèle 25
I.3.3.1 - Modèle de mémoire partagée (sans threads) 26
I.3.3.2 - Modèle de threads 26
I.3.3.3 - Modèle de mémoire distribuée/passage de messages . 27
I.3.3.4 - Modèle parallèle de données 29
I.3.3.5 - Modèle hybride 29
I.3.3.6 - Modèles SPMD et MPMD 29
I.3.4 - Environnement de programmation parallèle 30
I.3.5 - Défis de la conception d'algorithmes parallèles 33
I.4 - Conclusion 34

Chapitre II ► **Généralités sur les modèles multi-temporels et état de l'art** **35**
II.1 - Introduction 35
II.2 - Généralités sur les modèles multi-temporels 36
II.2.1 - Cadre formel et definition modeles temporels 36
II.2.2 - L'intérêt de l'extraction des modèles multi-temporels 41
II.2.3 - Défis de l'extraction des modèles multi-temporels 41
II.3 - État de l'art 43
II.3.1 - L'algorithme Apriori 43
II.3.1.1 - Contexte et origine 43
II.3.1.2 - Principe de fonctionnement 43
II.3.1.3 - Pseudocode de l'algorithme Apriori 43
II.3.1.4 - Explication détaillée avec exemple 44
II.3.2 - L'algorithme ARMADA 46
II.3.2.1 - Contexte et origine 46
II.3.2.2 - Principe de fonctionnement 47

II.3.2.3 - Pseudocode de l'algorithme ARMADA 47
II.3.3 - L'algorithme Discovering During-Temporal Patterns (DTP) . . 48
II.3.3.1 - Principe de fonctionnement 48
II.3.3.2 - Pseudocode de l'algorithme DTP 49
II.3.4 - Algorithme proposé par Xiaoxiao et al. [1] 50
II.3.4.1 - Principe de fonctionnement 50
II.3.4.2 - Pseudocode de l'algorithme 51
II.3.4.3 - Exemple de relation temporelle 52
II.3.4.4 - Prédicats temporels 52
II.3.4.5 - Faits temporels (augmentation et diminution) 53
II.4 - Limites de recherche 53
II.5 - Conclusion 54

Chapitre III ► **Extension des Prédicats, Utilisation des Matrices et Algorithme Parallèle pour l'Extraction des modèles multi-temporels** **55**
III.1 - Introduction 55
III.2 - Amélioration des prédicats 56
III.3 - Méthode de prétraitement des données 58
III.4 - Amélioration de l'algorithmiques d'extraction des modèles multi-temporels 61
III.4.1 - Utilisation de matrices 63
III.4.2 - Parallélisation de l'algorithme 67
III.5 - Expérimentations et résultats 70
III.5.1 - Analyse des prédicats temporels 70
III.5.2 - Optimisation de l'algorithme 71
III.6 - Synthèse 74

Conclusion générale **75**
Problématique étudiée et les choix méthodologiques 75
Analyse critique des résultats obtenus 76
Quelques perspectives 76

Bibliographie **78**

LISTE DES SYMBOLES

C_A	Contrainte de fenêtre pour l'événement e_A
t_ϕ	Intervalle de temps effectif pour un motif temporel ϕ
S	Ensemble d'états
R	Prédicat temporel
E	Ensemble des événements
Φ_k	Motif temporel de longueur k
$T(\phi)$	Ensemble des intervalles de temps effectifs pour un motif temporel ϕ
D_T	Base de données temporelle

LISTE DES ACRONYMES

ARIMA	AutoRegressive Integrated Moving Average ;
CNN	Convolutional Neural Networks ;
DBSCAN	Density-Based Spatial Clustering of Applications with Noise ;
GPU	Graphics Processing Unit ;
GSP	Generalized Sequential Pattern ;
KDD	Knowledge Discovery in Databases ;
K-NN	k-Nearest Neighbors ;
MLP	Multilayer Perceptron ;
MPI	Message Passing Interface ;
MPMD	Multiple Program Multiple Data ;
OpenMP	Open Multi-Processing ;
PrefixSpan	Prefix-projected Sequential Pattern Mining ;
SPMD	Single Program Multiple Data ;
SVM	Support Vector Machines ;

LISTE DES TABLEAUX

I - Base de données de transactions 15
II - Base de données de transactions 16

III - Exemple de transactions dans un supermarché 44
IV - 1-itemsets fréquents 45
V - 2-itemsets fréquents 45
VI - 3-itemsets fréquents 45

VII - Résultat tableau temporelle nettoyé 60
VIII - Comparaison des intervalles de temps 62
IX - Models temporels au niveau de l'industrie 70
X - Models temporels au niveau du marché 70
XI - Algorithme Matricielle - Accélération en fonction du nombre de threads 73

TABLE DES FIGURES

1 - Les trois phases du processus d'extraction de connaissances à partir des données [2] . . . 7
2 - Algorithme des arbres de décision [3] . . . 8
3 - Exemple de réseaux de neurones [4] . . . 9
4 - Algorithme des SVM(Supports Vectors Machine) [5] . . . 10
5 - Algorithme de K-means [6] . . . 12
6 - Exemple d'algorithme de Apriori . . . 13
7 - Classification de Flynn.[7] . . . 22
8 - Classification MIMD de Reina. [8] . . . 24
9 - Réseaux d'interconnexion statique. [9] . . . 25
10 - Modèle de mémoire partagée [10] . . . 27
11 - Modèle de threads [10] . . . 28
12 - Modèle de mémoire distribuée [10] . . . 28
13 - Modèle parallèle de données [10] . . . 29
14 - Modèle hybride avec MPI et OpenMP [10] . . . 30
15 - Modèle SPMD [10] . . . 30
16 - Modèle MPMD [10] . . . 30

17 - Exemple d'une base de données temporelle [1] . . . 37

18 - Relations temporelles selon les intervalles de temps d'Allen [11] . . . 57
19 - Chargement des données brutes . . . 58
20 - Conversion des données en événements . . . 59
21 - Conversion des données en événements . . . 59
22 - Debut code source . . . 60
23 - Fin code source . . . 61
24 - Étapes de l'algorithme . . . 68
25 - Temps d'exécution en fonction du nombre d'états. . . . 71
26 - Temps d'exécution en fonction du nombre d'états sur différents threads. 72
27 - Comparaison des Algorithmes (Xiaoxiao vs Final) . . . 73

RÉSUMÉ

La découverte des modèles multi-temporels est essentielle pour analyser des séquences d'événements ordonnés dans le temps, particulièrement dans des domaines tels que la finance où des volumes massifs de données sont générés. Identifier des modèles multi-temporels comme "le prix de l'action A augmente avant que le prix de l'action B augmente" peut offrir des aperçus précieux pour anticiper les mouvements du marché et prendre des décisions éclairées. Les algorithmes traditionnels d'extraction de modèles multi-temporels, bien que puissants, présentent des limitations en termes de granularité des prédicats et de temps de calcul. Inspirés par les travaux de Xiaoxiao Kong et al., nous avons étendu les prédicats temporels en intégrant les intervalles de temps d'Allen, offrant ainsi une représentation plus fine des relations entre événements. De plus, nous avons adopté des techniques de vectorisation et de parallélisation en utilisant le multithreading pour améliorer l'efficacité des calculs. La vectorisation a été utilisée pour le calcul du support, permettant de traiter efficacement les opérations de comparaison, tandis que la parallélisation a été appliquée au bloc de détection des modèles multi-temporels, exploitant les architectures de calculs modernes pour réduire le temps d'exécution. Nos expérimentations, menées sur une base de données financière de grande envergure, ont démontré des améliorations significatives en termes de précision et de rapidité d'exécution des algorithmes. L'algorithme parallélisé sur 4, 8, 16, 32 et 64 cœurs a montré des gains de performance allant jusqu'à 7,5 fois plus rapide que l'algorithme séquentiel, tandis que l'algorithme basé sur les matrices a permis d'augmenter la vitesse de calcul de 8,4 fois. En combinant ces améliorations, nous avons obtenu une augmentation de performance totale de 15,9 fois par rapport à l'algorithme de Xiaoxiao, rendant ainsi le processus d'extraction de modèles multi-temporels beaucoup plus rapide et efficace.

Mots clés : Modèles multi-temporels, Prédicats temporels, Vectorisation, Parallélisation, Bases de données financières, Mémoire partagée.

Extension of Temporal Predicates, Utilization of Matrices, and Parallelization for Extracting Multi-Temporal Patterns in Temporal Databases

ABSTRACT

The discovery of multi-temporal patterns is essential for analyzing sequences of events ordered over time, particularly in fields such as finance where massive volumes of data are generated. Identifying multi-temporal patterns like "the price of stock A increases before the price of stock B increases" can offer valuable insights for anticipating market movements and making informed decisions. Traditional algorithms for extracting multi-temporal patterns, while powerful, have limitations in terms of predicate granularity and computation time. Inspired by the work of Xiaoxiao Kong et al., we have extended temporal predicates by integrating Allen's interval algebra, offering a finer representation of relationships between events. Additionally, we have adopted vectorization and parallelization techniques using multithreading to improve computational efficiency. Vectorization was used for support computation, allowing efficient handling of comparison operations, while parallelization was applied to the multi-temporal pattern detection block, leveraging modern computing architectures to reduce execution time. Our experiments, conducted on a large-scale financial database, demonstrated significant improvements in both accuracy and execution speed of the algorithms. The parallelized algorithm on 4, 8, 16, 32, and 64 cores showed performance gains of up to 7.5 times faster than the sequential algorithm, while the matrix-based algorithm increased computation speed by 8.4 times. By combining these improvements, we achieved a total performance increase of 15.9 times compared to Xiaoxiao's algorithm, making the process of extracting multi-temporal patterns much faster and more efficient.

Keywords : Multi-temporal patterns, Temporal predicates, Vectorization, Parallelization, Financial databases, Shared memory.

INTRODUCTION GÉNÉRALE

SOMMAIRE

Contexte du travail 1
Problématique étudiée 2
Objectif 2
Contribution 3
Plan de travail 3

Contexte du travail

L'extraction de motifs multi-temporels est un aspect crucial de la fouille de données (Data Mining). Cette discipline consiste à analyser des séquences de données temporelles afin de découvrir des motifs récurrents qui peuvent fournir des aperçus précieux pour la prise de décision dans divers domaines. La fouille de données temporelles s'applique particulièrement aux bases de données financières, où les comportements des marchés peuvent être influencés par des événements interconnectés sur des périodes spécifiques. D'autres domaines d'application incluent les bases de données biologiques et les bases de données météorologiques.

Les motifs multi-temporels permettent de mettre en évidence des corrélations complexes entre différents événements qui se produisent à des moments distincts. Par exemple, une règle temporelle peut prendre la forme : "Si l'événement A se produit, alors l'événement B est susceptible de se produire après une période donnée". Ces règles sont particulièrement utiles pour prédire des comportements futurs et pour comprendre les dynamiques sous-jacentes des systèmes complexes.

L'amélioration de l'efficacité des algorithmes d'extraction de motifs multi-temporels est essentielle pour gérer le volume croissant de données temporelles. Les approches traditionnelles, telles que l'algorithme présenté dans les travaux de Kong et al. (2010) [1], ont jeté les bases de la découverte de motifs multi-temporels, mais

nécessitent des optimisations pour être applicables à des ensembles de données de grande taille. c'est sur ce dernier que repose l'intégralité de notre travail.

Problématique étudiée

L'extraction de modèles multi-temporels à partir de grandes bases de données pose des défis significatifs en termes de complexité et de performance. Les algorithmes actuels, bien qu'efficaces pour des volumes de données modestes, peinent à gérer l'ampleur des données générées dans des applications modernes telles que les bases de données financières, biologiques, météorologiques. De plus, les prédicats temporels classiques utilisés pour la découverte de motifs peuvent être ambigus pour la recherche des relations temporelles.

Ainsi, nous pouvons nous poser la question de savoir : **Comment améliorer les prédicats temporels pour capturer des relations temporelles sémantiquement précises entre les états et augmenter la vitesse d'exécution des algorithmes de découverte de modèles multi-temporels ?**

Objectif

L'extraction des motifs multi-temporels est un sujet qui a fait l'objet de nombreuses études et publications (thèses de doctorat, articles), enrichissant nos idées, nourrissant nos réflexions et orientant nos pensées pour finalement poser les bases de notre travail. Sur ces bases, nous nous sommes appliqués à améliorer les performances en temps CPU des algorithmes existants et à développer de nouveaux algorithmes basés sur des concepts élaborés par nos prédécesseurs.

Pour y parvenir, nous avons employé le processus suivant :

— **Enrichissement des prédicats temporels** : Proposer de nouveaux prédicats temporels pour capturer des relations temporelles sémantiquement précises entre les événements. Ces nouveaux prédicats permettront d'exprimer des relations temporelles plus spécifiques et moins ambiguës, améliorant ainsi la qualité des motifs découverts.

— **Amélioration de la vitesse d'exécution** : Optimiser le temps d'exécution des algorithmes de découverte de modèles multi-temporels en utilisant des matrices pour le calcul du support, et paralléliser les opérations de génération des modèles multi-temporels. Cette approche permet de structurer les données de manière plus efficace pour les calculs du support et d'exploiter

les capacités de calcul parallèle, rendant ainsi le processus de découverte de modèles plus scalable et adapté aux grands volumes de données.

— **Implémentation et expérimentation** : Tester un jeux de données sur l'approche proposée par l'article [1] et notre approche pour évaluer la vitesse d'exécution et la qualité des relations temporelles sémantiquement précises.

Contribution

Dans ce travail de recherche, nous proposons une approche innovante pour l'extraction de motifs temporels. Nous enrichissons les prédicats temporels en utilisant les intervalles de temps d'Allen et améliorons la vitesse d'exécution des algorithmes de découverte de motifs temporels en parallélisant les opérations de génération des modèles multi-temporels. et en utilisant des matrices pour le calcul du support.

Plan de travail

Notre document est réparti en trois chapitres comme suit :

Chapitre 1. Généralité sur la fouille de données et sur la parallélisation : Avant de s'attaquer au cœur du sujet, il est important de s'assurer de la compréhension des éléments de base nécessaires pour appréhender les notions abordées par la suite. Ce chapitre sera consacré à la compréhension de la fouille de données et de la parallélisation. Nous aborderons les concepts clés de la fouille de données, les techniques de prétraitement des données, et les principaux algorithmes de fouille. Ensuite, nous nous concentrerons sur la parallélisation, en discutant de son importance, des différentes approches et techniques de parallélisation, ainsi que des défis associés à la mise en œuvre de solutions parallèles.

Chapitre 2. Généralité sur les modèles multi-temporels et l'état de l'art : Ce chapitre permet de comprendre les modèles multi-temporels, ainsi nous examinerons les travaux précédents et les algorithmes performants déjà publiés par des chercheurs. L'objectif est de fournir une vue d'ensemble des avancées récentes et des meilleures pratiques dans le domaine des modèles multi-temporels. Nous analyserons également les défis actuels et les lacunes des approches existantes, afin d'identifier les opportunités d'amélioration.

Chapitre 3. Contribution : Dans ce chapitre, nous présenterons notre contribution spécifique au domaine de l'extraction de modèles multi-temporels. Nous dé-

taillerons notre approche innovante qui enrichit les prédicats temporels et améliore la vitesse d'exécution des algorithmes en utilisant des matrices pour le calcul du support, ainsi qu'en parallélisant les opérations de génération des modèles multi-temporels. Nous discuterons des résultats obtenus lors de nos expérimentations et comparerons les performances de notre méthode avec celles des algorithmes existants.

I

CHAPITRE

GÉNÉRALITÉS SUR LA FOUILLE DE DONNÉES ET SUR LA PARALLÉLISATION

SOMMAIRE

I.1 - Introduction 5
I.2 - Généralités sur la fouille de données 6
I.3 - Généralités sur la parallélisation 19
I.4 - Conclusion 34

I.1. Introduction

La fouille de données, ou découverte de connaissances à partir de données (KDD), est une discipline à la frontière entre les statistiques, l'intelligence artificielle et l'informatique. Elle vise à extraire des informations significatives et des motifs cachés à partir de grands ensembles de données. Dans un monde où les volumes de données augmentent de manière exponentielle, la capacité à analyser ces données et à en tirer des connaissances exploitables est devenue cruciale pour de nombreux secteurs, notamment la finance, la santé, le marketing et les sciences sociales. La fouille de données temporelles se distingue par sa capacité à prendre en compte la dimension temporelle des données, permettant ainsi de découvrir des motifs et des tendances qui évoluent dans le temps. Cette approche est particulièrement pertinente dans le domaine financier, où les comportements des marchés sont influencés par une multitude d'événements interconnectés dans le temps. Les modèles temporels sont des outils puissants pour capturer et analyser ces relations temporelles complexes, permettant non seulement de comprendre les dynamiques

passées, mais aussi de prédire les comportements futurs, offrant ainsi un avantage stratégique pour les décideurs. Les modèles multi-temporels, en particulier, fournissent une vue plus riche des interactions entre les événements en intégrant plusieurs prédicats temporels. Ce chapitre se propose de fournir une vue d'ensemble des concepts fondamentaux de la fouille de données et des modèles temporels. Il abordera d'abord les aspects généraux de la fouille de données, y compris ses objectifs, ses techniques et ses applications. Ensuite, il explorera les différents types de modèles temporels utilisés en fouille de données, en mettant l'accent sur les motifs séquentiels et les motifs fréquents temporels. Enfin, nous discuterons de l'importance de l'extraction des modèles temporels et des défis associés à cette tâche.

I.2. Généralités sur la fouille de données

I.2.1. Définition et historique

La fouille de données, ou Data Mining, a été définie en 1996 par Usama Fayyad, Gregory Piatetsky-Shapiro et Padhraic Smyth comme "une nouvelle génération de théories computationnelles et d'outils pour aider les humains à extraire des informations utiles (connaissances) des volumes croissants de données numériques" [12]. Ce domaine se caractérise par quatre objectifs principaux :

- Extraction des informations cachées : Découvrir des modèles et des connaissances enfouies dans les données.
- Gestion de grandes quantités de données : Aborder les défis posés par l'énorme volume de données.
- Transformation des données brutes en connaissances : Convertir des données non structurées en informations exploitables et expertes.
- Fourniture de connaissances nouvelles et utiles : Générer des insights précieux, valides et applicables pour les utilisateurs.

Le "processus de fouille de données" combine différentes techniques issues de disciplines telles que l'analyse de données, l'intelligence artificielle et l'apprentissage automatique. Contrairement aux données expérimentales couramment utilisées en statistique, la fouille de données se concentre sur l'analyse de données d'observation collectées pour d'autres raisons que l'analyse elle-même. Cette approche permet de traiter de grandes quantités d'informations et d'en extraire des connaissances pertinentes.

Un processus typique de fouille de données comporte trois étapes principales :

1. **Préparation des données** : Nettoyage des données pour éliminer les bruits et les incohérences, intégration cohérente de données provenant de sources multiples, normalisation et réduction des données par agrégation ou élimination.
2. **Fouille de données proprement dite** : Utilisation de différentes techniques pour l'analyse exploratoire ou la recherche de modèles et de motifs.
3. **Mise en forme des résultats** : Utilisation de techniques de représentation des connaissances et de visualisation pour faciliter l'interprétation des résultats par les utilisateurs.

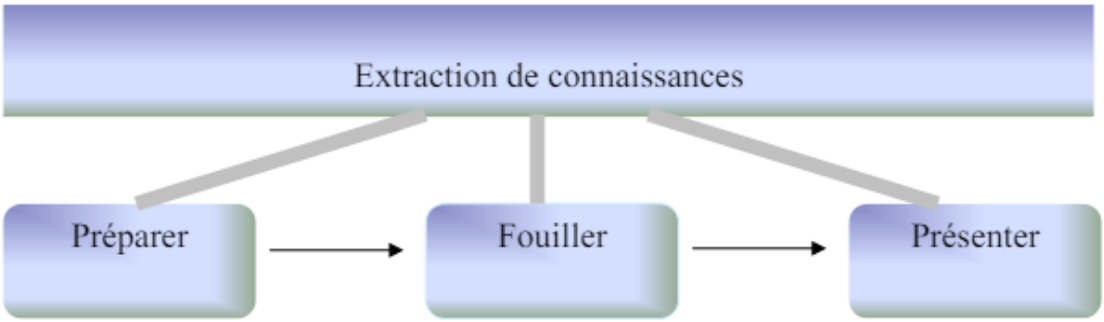


FIGURE 1 – *Les trois phases du processus d'extraction de connaissances à partir des données [2]*

La fouille de données explore et analyse de grandes quantités de données pour découvrir des modèles, des tendances et des relations cachées. Elle utilise des techniques statistiques, mathématiques et informatiques pour extraire des informations utiles à partir de données brutes et est appliquée dans divers domaines tels que la finance, le marketing, la santé et les sciences sociales. Par exemple, dans le domaine financier, la fouille de données peut aider à identifier des tendances de marché et à améliorer les décisions d'investissement.

Cependant, la fouille de données soulève des questions éthiques, notamment en ce qui concerne la confidentialité et la protection des données personnelles. Les entreprises doivent donc se conformer aux lois en vigueur sur la protection des données lorsqu'elles collectent et analysent ces informations.

I.2.2. Les techniques et les algorithmes de fouille de données

I.2.2.1. La classification

La classification en fouille de données est définie comme le processus d'attribution de classes ou de catégories à un ensemble de données basé sur des caractéristiques communes. Selon l'article de Han, Kamber et Pei [13] la classification

vise à prédire avec précision la classe cible des objets dont l'étiquette de classe est inconnue. Ce processus utilise divers algorithmes pour analyser les données et identifier les motifs qui peuvent aider à classer les données efficacement. Parmi les algorithmes de classification les plus couramment utilisés, on trouve les arbres de décision, les réseaux de neurones, les machines à vecteurs de support (SVM), les k-plus proches voisins (k-NN), la régression logistique

1. **Arbres de décision** :Les arbres de décision sont des modèles prédictifs qui utilisent un ensemble de règles dérivées des caractéristiques des données pour prendre des décisions. Ils sont populaires en raison de leur simplicité et de leur interprétabilité. Un exemple classique est l'algorithme C4.5, qui génère des arbres de décision à partir de données d'entraînement en utilisant le concept d'entropie pour évaluer les décisions. Cet algorithme construit un arbre en sélectionnant les attributs qui réduisent le plus l'incertitude à chaque niveau.

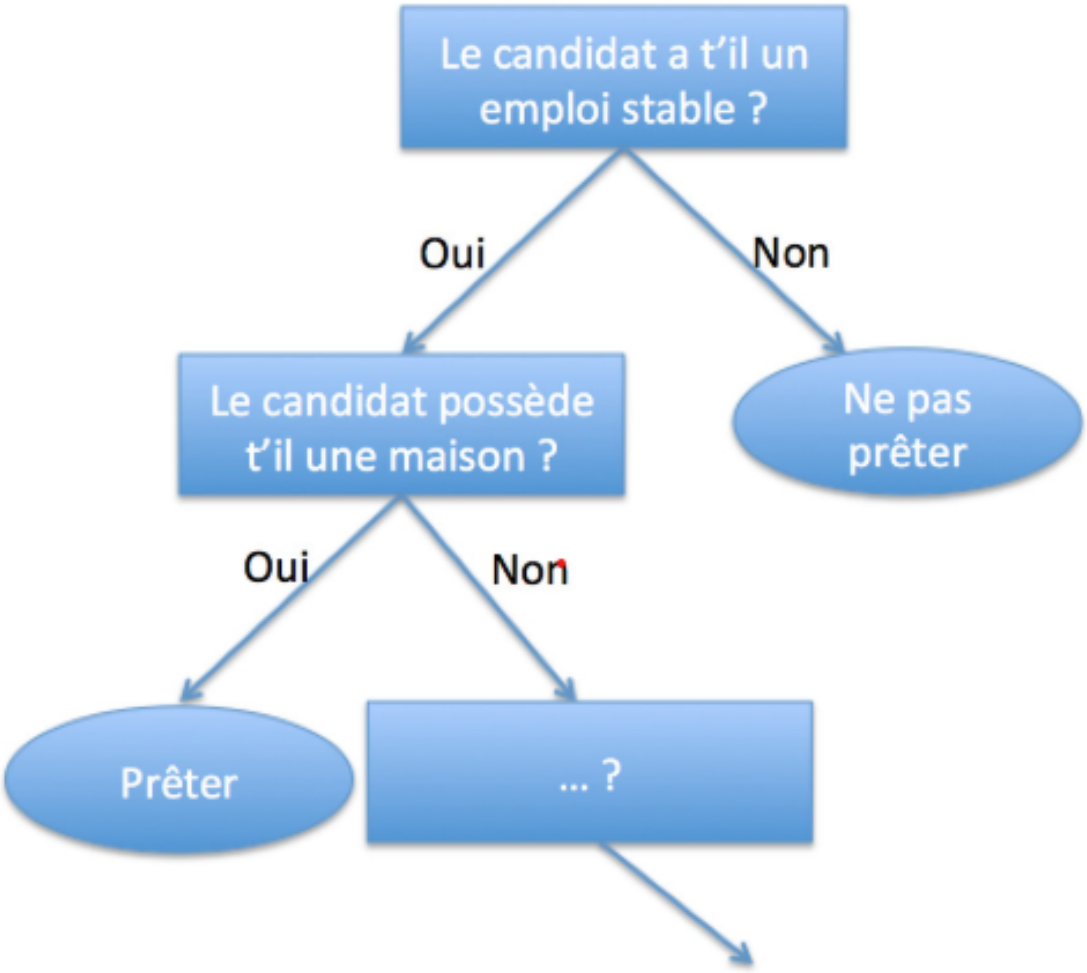


FIGURE 2 – *Algorithme des arbres de décision [3]*

2. **Réseaux de neurones** :Les réseaux de neurones artificiels sont des modèles inspirés du cerveau humain, capables d'apprendre des représentations complexes des données grâce à plusieurs couches de neurones artificiels. Le perceptron multicouche (MLP) est un exemple typique, utilisant des couches cachées pour capturer des motifs complexes dans les données. Les réseaux de neurones convolutifs (CNN) sont également largement utilisés, notamment pour les tâches de reconnaissance d'images.

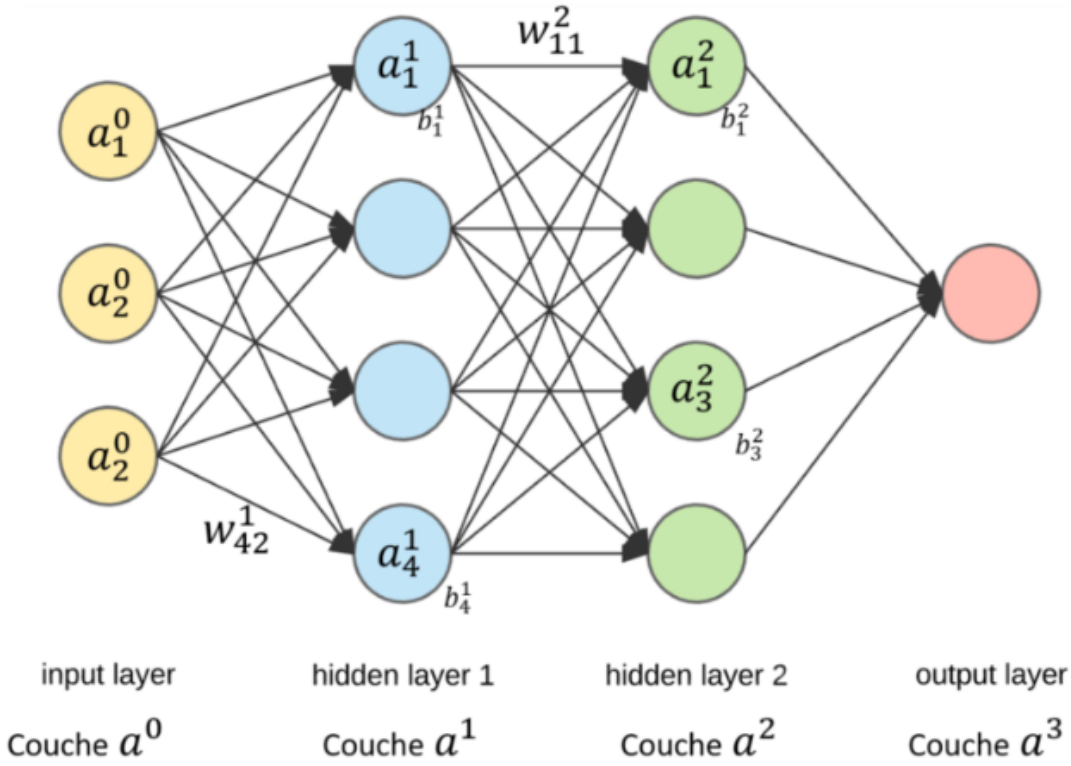


FIGURE 3 – *Exemple de réseaux de neurones [4]*

3. **Machines à vecteurs de support (SVM)** : Une méthode d'apprentissage supervisé utilisée pour la classification et la régression. L'algorithme trouve un hyperplan optimal qui sépare les données en deux classes en maximisant la marge entre l'hyperplan et les points les plus proches de chaque classe. Les SVM peuvent également utiliser des noyaux pour transformer les données dans un espace de dimension supérieure afin de mieux séparer les classes. Ils sont largement utilisés dans la reconnaissance d'image, la classification de texte et la bio-informatique.

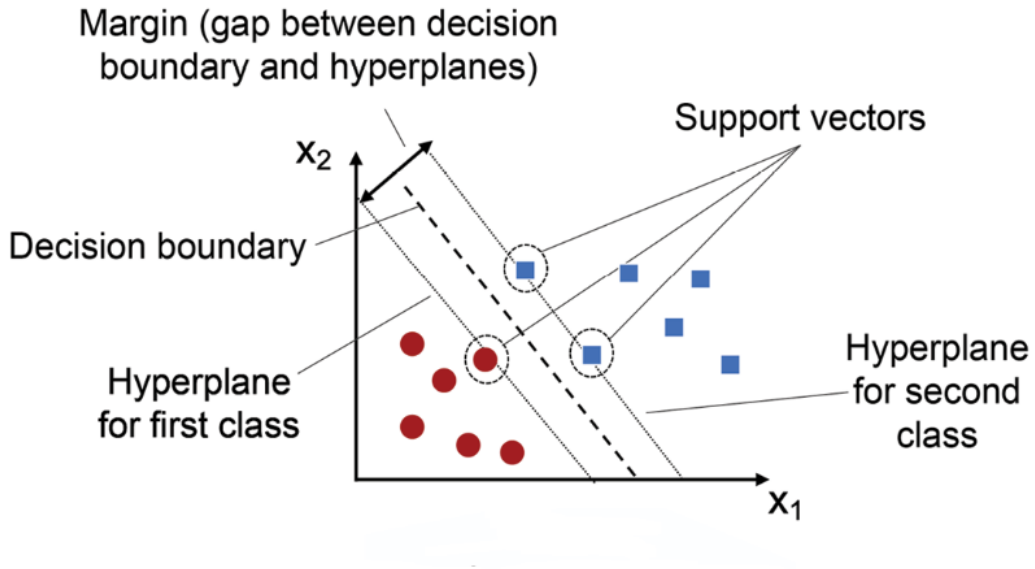


FIGURE 4 – *Algorithme des SVM(Supports Vectors Machine) [5]*

I.2.2.2. La régression

La régression est une méthode d'apprentissage supervisé utilisée pour modéliser la relation entre une variable dépendante et une ou plusieurs variables indépendantes. Elle est largement utilisée pour les prévisions et les prédictions dans divers domaines, y compris la finance, l'économie, et les sciences sociales. Selon Seber et Lee [14], la régression permet de comprendre et de quantifier les relations entre les variables, aidant ainsi à faire des prédictions informées et à prendre des décisions basées sur les données.

1. **Régression linéaire** : La régression linéaire est une méthode qui modélise la relation entre une variable dépendante et une ou plusieurs variables indépendantes en ajustant une ligne droite (ou hyperplan) aux données. Elle est utilisée pour les prévisions et les tendances. La régression linéaire simple implique une seule variable indépendante, tandis que la régression linéaire multiple peut inclure plusieurs variables indépendantes. Cette méthode est souvent utilisée en économie pour prédire les tendances du marché ou en biostatistique pour évaluer l'impact de divers facteurs sur la santé [14].
2. **Régression polynomiale** : La régression polynomiale étend la régression linéaire en permettant que la relation entre les variables dépendantes et indépendantes soit modélisée sous la forme d'un polynôme. Cela permet de capturer les relations non linéaires dans les données. Par exemple, un modèle de régression polynomiale pourrait être utilisé pour prédire les ventes en

fonction du temps, où la relation entre le temps et les ventes n'est pas linéaire mais suit une courbe [15].

3. **Régression Ridge** : La régression Ridge est une technique de régularisation qui modifie la fonction de coût de la régression linéaire pour inclure une pénalité pour la taille des coefficients. Cela aide à prévenir le surapprentissage et améliore la généralisation du modèle. Cette méthode est particulièrement utile lorsque les données ont des multicolinéarités, c'est-à-dire lorsque les variables indépendantes sont fortement corrélées entre elles [16].
4. **Régression Lasso** : La régression Lasso est une autre technique de régularisation qui ajoute une pénalité pour la somme des valeurs absolues des coefficients à la fonction de coût. Cela peut entraîner la suppression de certains coefficients, conduisant à des modèles plus simples et interprétables. La régression Lasso est particulièrement utile pour la sélection de variables, car elle peut réduire certains coefficients à zéro, éliminant ainsi les variables non pertinentes du modèle [17].

I.2.2.3. Le clustering

Le clustering, également connu sous le nom d'analyse de regroupement, est une technique de fouille de données non supervisée qui vise à regrouper un ensemble de données en clusters ou groupes de données similaires. Selon l'article de Jain, Murty et Flynn [18], le clustering est utilisé pour découvrir des structures sous-jacentes dans les données sans avoir recours à des étiquettes prédéfinies.

1. **K-means** : K-means est une méthode de clustering populaire qui partitionne les données en un nombre prédéfini de clusters (k). L'algorithme commence par sélectionner k centres aléatoires pour chaque cluster, puis attribue chaque point de données au centre le plus proche. Ensuite, les centres sont recalculés en utilisant la moyenne des points appartenant à chaque cluster. Ce processus est répété jusqu'à ce que les centres ne bougent plus ou que le nombre maximum d'itérations soit atteint. L'algorithme K-means est largement utilisé dans l'analyse de données et la segmentation de marché [19].

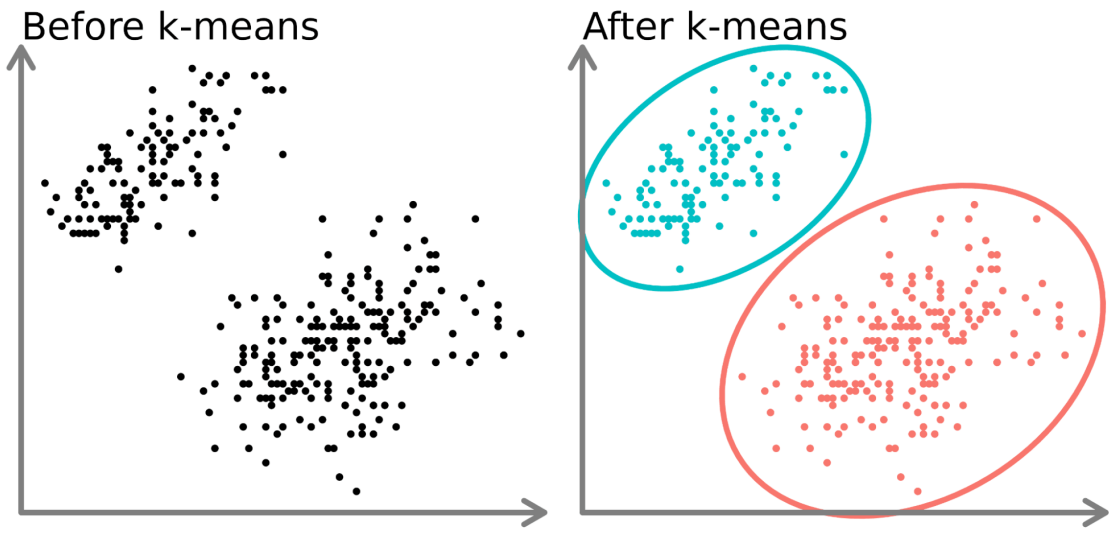


FIGURE 5 – *Algorithme de K-means [6]*

2. **DBSCAN** : DBSCAN (Density-Based Spatial Clustering of Applications with Noise) est un algorithme de clustering basé sur la densité qui peut identifier des clusters de forme arbitraire et traiter les points bruyants (noise points). DBSCAN fonctionne en sélectionnant des points de données comme noyaux et en étendant les clusters à partir de ces noyaux en fonction de la densité locale des points. Cet algorithme est particulièrement utile pour détecter des clusters dans des ensembles de données contenant du bruit

I.2.2.4. La règles d'association

Les règles d'association sont des techniques de fouille de données utilisées pour identifier des relations intéressantes entre des variables dans de grandes bases de données. Ces règles sont souvent formulées sous la forme "Si X, alors Y", indiquant une association forte entre l'ensemble d'items X et Y. Agrawal et al. (1993) sont souvent cités pour leur travail pionnier sur ces méthodes dans des bases de données transactionnelles [20].

Support et confiance :

— Le *support* d'une règle indique la fréquence à laquelle les items de la règle apparaissent ensemble dans les transactions de la base de données. Il est crucial pour mesurer l'importance statistique de la règle dans la base de données.

— La *confiance* est une mesure de la fiabilité de l'inférence de la règle. Elle est définie comme la probabilité conditionnelle que Y soit acheté lorsque X est acheté [21].

Algorithme Apriori : L'algorithme Apriori, développé par Agrawal et Srikant (1994), est l'un des algorithmes les plus connus pour extraire des règles d'association. Il repose sur une approche itérative où des ensembles d'items fréquents sont étendus un item à la fois, et les groupes d'items candidats sont testés contre les données. L'efficacité de l'algorithme repose sur la propriété que tous les sous-ensembles d'un ensemble fréquent doivent également être fréquents [22].

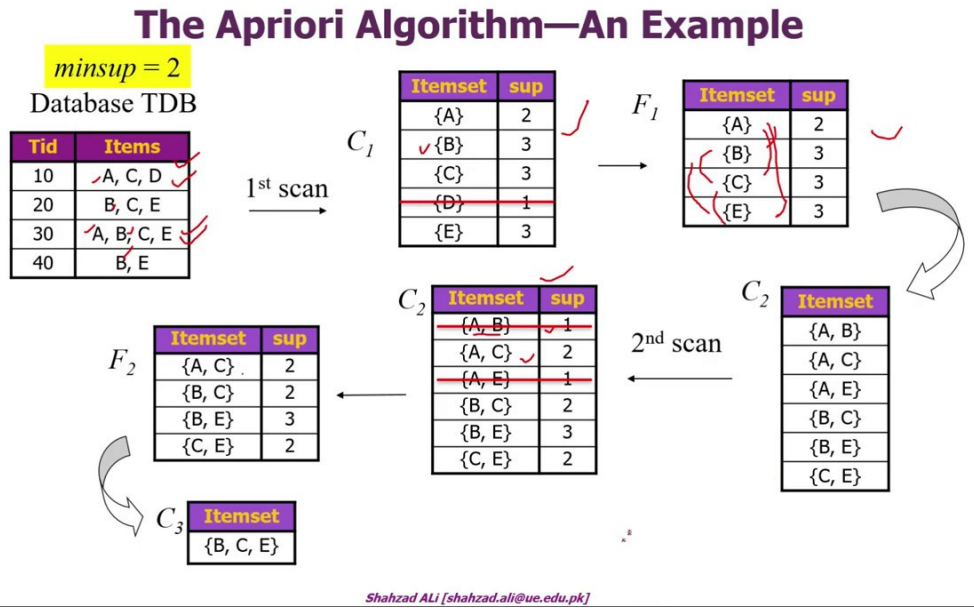


FIGURE 6 – *Exemple d'algorithme de Apriori*

Applications pratiques : Les règles d'association sont largement appliquées dans de nombreux domaines, notamment :

— Les analyses de panier de marché pour recommander des produits croisés.
— Les systèmes de recommandation dans les médias numériques.
— La détection de comportements frauduleux par analyse des transactions inhabituelles.

Les règles d'association ne se limitent pas à la vente au détail ; elles sont également utilisées dans le secteur de la santé pour analyser les schémas de prescription, et en finance pour identifier les schémas de transactions suspectes.

I.2.2.5. Analyse de séries temporelles

L'analyse de séries temporelles examine des données collectées à intervalles réguliers, offrant des insights pour la prévision dans des domaines tels que l'économie, la météorologie et la finance.

Définition et Principes : Les séries temporelles sont des ensembles de données séquentielles enregistrées à des moments successifs, permettant l'analyse et

la prédiction des tendances. Box et Jenkins, dans leur développement des modèles ARIMA, ont souligné que ces outils sont essentiels pour comprendre les dynamiques temporelles, intégrant des approches auto-régressives, d'intégration et de moyenne mobile [23].

Techniques de Modélisation :

— *Modèles ARIMA* : Comme le soulignent Han, Kamber et Pei, ces modèles sont efficaces pour prévoir les valeurs futures des séries qui montrent des tendances et des cycles, bien adaptés aux données non saisonnières [13].

— *Lissage Exponentiel* : Hyndman a démontré que les modèles de lissage exponentiel sont particulièrement efficaces pour les prévisions à court terme quand les données présentent des tendances et une saisonnalité, offrant une alternative moins complexe que les modèles ARIMA [24].

I.2.2.6. Les motifs séquentiels

Les motifs séquentiels consistent à identifier des sous-séquences fréquentes dans une base de données séquentielle. Deux des algorithmes les plus populaires pour l'extraction de motifs séquentiels sont GSP (Generalized Sequential Pattern) et PrefixSpan (Prefix-projected Sequential Pattern Mining). Ces algorithmes diffèrent dans leur approche pour identifier les séquences fréquentes.

Algorithme GSP

L'algorithme GSP, proposé par Srikant et Agrawal en 1996 [25], est basé sur une approche itérative pour découvrir les motifs séquentiels fréquents. Il utilise une méthode de recherche par extension de séquences candidates et par prunes des séquences non fréquentes. Le processus se déroule comme suit :

1. **Initialisation :** Trouver toutes les 1-séquences fréquentes.
2. **Génération de candidats :** Générer des séquences candidates de longueur $k+1$ à partir des séquences fréquentes de longueur k.
3. **Évaluation des candidats :** Compter les occurrences des séquences candidates dans la base de données.
4. **Filtrage :** Garder uniquement les séquences candidates fréquentes, c'est-à-dire celles dont le support est supérieur à un seuil minimum.
5. Répéter les étapes 2 à 4 jusqu'à ce qu'il n'y ait plus de séquences candidates fréquentes.

Exemple : Considérons une base de données de transactions suivante :

ID Transaction	Séquence
1	{A, B}, {C}
2	{A}, {C, D}
3	{A, B}, {D}
4	{B}, {C}, {D}

TABLE I – *Base de données de transactions*

Étape 1 : Initialisation

— Séquences fréquentes de longueur 1 (L_1) : {A}, {B}, {C}, {D}

Étape 2 : Génération de candidats (C_2)

— Génération des 2-séquences : {A, B}, {A, C}, {A, D}, {B, C}, {B, D}, {C, D}

Étape 3 : Évaluation des candidats

— Comptage des occurrences dans la base de données :
  — {A, B} : 2 occurrences (Transactions 1, 3)
  — {A, C} : 2 occurrences (Transactions 1, 2)
  — {A, D} : 1 occurrence (Transaction 2)
  — {B, C} : 2 occurrences (Transactions 1, 4)
  — {B, D} : 1 occurrence (Transaction 3)
  — {C, D} : 2 occurrences (Transactions 2, 4)

Étape 4 : Filtrage

— Séquences fréquentes de longueur 2 (L_2) : {A, B}, {A, C}, {B, C}, {C, D}

Étape 2 : Génération de candidats (C_3)

— Génération des 3-séquences : {A, B, C}, {A, C, D}, {B, C, D}

Étape 3 : Évaluation des candidats

— Comptage des occurrences dans la base de données :
  — {A, B, C} : 1 occurrence (Transaction 1)
  — {A, C, D} : 1 occurrence (Transaction 2)
  — {B, C, D} : 1 occurrence (Transaction 4)

Étape 4 : Filtrage

— Séquences fréquentes de longueur 3 (L_3) : aucune

Résultat final :

— Motifs séquentiels fréquents : {A}, {B}, {C}, {D}, {A, B}, {A, C}, {B, C}, {C, D}

Algorithme PrefixSpan

L'algorithme PrefixSpan, proposé par Pei et al. en 2001 [26], est une méthode de fouille de motifs séquentiels qui se base sur le concept de projection de préfixes. Contrairement à GSP, qui génère des candidats et effectue des scans répétés de la base de données, PrefixSpan projette la base de données sur des préfixes croissants et mine les motifs séquentiels de manière récursive.

Étapes de l'algorithme PrefixSpan :

1. **Initialisation :** Trouver toutes les 1-séquences fréquentes.
2. **Projection :** Pour chaque 1-séquence fréquente, projeter la base de données pour ne conserver que les suffixes des séquences où le préfixe est présent.
3. **Fouille récursive :** Appliquer l'algorithme de manière récursive sur chaque base de données projetée.

Exemple : Utilisons la même base de données de transactions que pour GSP.

ID Transaction	Séquence
1	{A, B}, {C}
2	{A}, {C, D}
3	{A, B}, {D}
4	{B}, {C}, {D}

TABLE II – *Base de données de transactions*

Étape 1 : Initialisation

— Séquences fréquentes de longueur 1 : {A}, {B}, {C}, {D}

Étape 2 : Projection

— **Projetion sur {A}** :
  — Base projetée : {B}, {C}, {B}, {C, D}, {B}, {D}
— **Projetion sur {B}** :
  — Base projetée : {C}, {C, D}, {D}
— **Projetion sur {C}** :
  — Base projetée : {D}
— **Projetion sur {D}** :
  — Base projetée : vide

Étape 3 : Fouille récursive

— **Pour {A}** :
  — {A, B} : Base projetée {C}, {C, D}, {D}
  — {A, C} : Base projetée {D}
  — {A, D} : Base projetée vide

— **Pour {B}** :
  — {B, C} : Base projetée {D}
  — {B, D} : Base projetée vide
— **Pour {C}** :
  — {C, D} : Base projetée vide

Résultat final :

— Motifs séquentiels fréquents : {A}, {B}, {C}, {D}, {A, B}, {A, C}, {B, C}, {C, D}

I.2.3. Les applications de la fouille de données

I.2.3.1. Secteurs d'application

La fouille de données est employée dans une multitude de secteurs, chacun exploitant cette technologie pour extraire des informations précieuses qui améliorent la prise de décision et optimisent les processus. Parmi les principaux secteurs d'application, on trouve :

— **Finance** : Utilisée pour analyser les comportements de transaction, détecter les fraudes, évaluer les risques et optimiser les portefeuilles d'investissement.
— **Santé** : Employée pour analyser les dossiers médicaux, prédire les épidémies, personnaliser les traitements et améliorer la gestion des ressources hospitalières.
— **Marketing** : Permet aux entreprises de comprendre les préférences des consommateurs, cibler les campagnes publicitaires, et optimiser les stratégies de vente et de distribution.
— **Sécurité Publique** : Utilisée pour analyser les données des incidents, améliorer la réponse aux urgences et planifier le déploiement des ressources de sécurité.

I.2.3.2. Cas d'usage concrets

Les techniques de fouille de données sont appliquées dans des contextes variés, démontrant leur flexibilité et leur efficacité :

— **Détection de fraude** : Dans le secteur bancaire, la fouille de données aide à identifier les schémas de transactions inhabituels qui peuvent indiquer une fraude, permettant ainsi une intervention rapide pour minimiser les pertes.
— **Prévention des réadmissions hospitalières** : Les hôpitaux utilisent la fouille de données pour identifier les patients présentant un risque élevé de réad-

mission et développer des stratégies de suivi personnalisées pour prévenir les retours prématurés.

— **Optimisation de la chaîne logistique** : Les entreprises manufacturières utilisent des techniques d'analyse de données pour étudier les séquences de production et de livraison. En examinant ces séquences, elles peuvent identifier les goulets d'étranglement, c'est-à-dire les étapes où des retards ou des inefficacités se produisent. Une fois ces goulets d'étranglement identifiés, elles peuvent mettre en place des mesures pour les éliminer ou les réduire, ce qui améliore l'efficacité globale de la chaîne logistique. Par exemple, cela peut inclure l'ajustement des calendriers de production, l'amélioration des processus de gestion des stocks ou l'optimisation des itinéraires de livraison.

— **Gestion de l'énergie** : Les fournisseurs d'énergie utilisent la fouille de données pour analyser les tendances de consommation et prévoir la demande énergétique future. Cette prévision permet d'optimiser la production d'énergie en ajustant la quantité d'énergie produite en fonction de la demande prévue, ce qui réduit le gaspillage et les coûts. En outre, la fouille de données aide à optimiser la distribution de l'énergie en identifiant les meilleures façons de distribuer l'énergie produite pour minimiser les pertes et les coûts de transport. Les stratégies de tarification dynamique sont également développées à l'aide de ces analyses. Elles permettent aux fournisseurs d'ajuster les prix de l'énergie en temps réel en fonction de l'offre et de la demande, encourageant ainsi une utilisation plus équilibrée de l'énergie et évitant les surcharges du réseau.

I.2.4. Les défis et les limitations de la fouille de données

La fouille de données, bien qu'extrêmement puissante, rencontre plusieurs défis et limitations qui peuvent compromettre son efficacité et son acceptabilité dans divers contextes. Ces défis comprennent des enjeux liés à la qualité des données, des problématiques éthiques et de confidentialité, ainsi qu'à la complexité computationnelle. Chacun de ces aspects nécessite une attention particulière pour garantir que les projets de fouille de données soient non seulement efficaces mais également éthiques et respectueux de la vie privée.

I.2.4.1. Qualité des données

La qualité des données est au cœur de la performance de la fouille de données. La précision, la complétude, et la pertinence des données collectées influencent directement la validité des analyses réalisées. Des données mal nettoyées, incom-

plètes ou erronées peuvent conduire à des conclusions inexactes, affectant négativement les décisions prises sur leur base. Le nettoyage des données, qui inclut la correction des erreurs, le remplissage des données manquantes et l'élimination des duplicatas, est donc une étape cruciale. Selon une étude de Kandel et al., les scientifiques passent jusqu'à 80% de leur temps à préparer et nettoyer les données avant d'effectuer toute analyse, ce qui souligne l'importance cruciale de ce processus [27].

I.2.4.2. Problèmes éthiques et de confidentialité

Avec l'accroissement de la collecte de données, notamment celles à caractère personnel, les problèmes éthiques et de confidentialité prennent une place prépondérante. La possibilité d'identifier indirectement des individus à partir de données anonymisées devient une préoccupation majeure. Les régulations telles que le GDPR tentent de s'attaquer à ces problèmes en imposant des normes strictes sur la collecte, le traitement, et le partage des données personnelles. Mayer-Schönberger et Cukier ont discuté de l'équilibre nécessaire entre l'innovation permise par la fouille de données et le respect des droits individuels, soulignant l'importance de développer des approches responsables pour gérer ces données [28].

I.2.4.3. Complexité computationnelle

Les algorithmes de fouille de données traitent souvent de vastes volumes de données, ce qui peut entraîner une complexité computationnelle significative. Cette complexité se manifeste non seulement dans le besoin de ressources informatiques puissantes mais aussi dans la nécessité de développer des algorithmes efficaces et optimisés. Les défis incluent la gestion de grandes bases de données, l'exécution de calculs intensifs, et l'application de modèles complexes. Wu et Kumar ont examiné comment ces aspects affectent la performance et la scalabilité des solutions de fouille de données, mettant en lumière les défis techniques à surmonter pour tirer pleinement parti de ces technologies [29].

I.3. Généralités sur la parallélisation

I.3.1. L'évolution des technologies et le besoin de parallélisation

À la fin des années 1980, de nombreuses innovations, telles que le World Wide Web, Google, Microsoft et Linux, ont entraîné la croissance et l'émergence de nombreux domaines dans la recherche aéronautique, spatiale, économique, médi-

cale, militaire, etc. Depuis lors, les technologies de l'information et de la communication ont connu une croissance exponentielle grâce au développement de divers matériaux et systèmes informatiques [30]. Ces technologies permettent d'accéder, de traiter, d'analyser, de stocker ou de transférer des informations sous différentes formes pour répondre aux besoins des utilisateurs.

En 2020, une étude sur les confinements imposés par le gouvernement aux citoyens européens en raison de la pandémie de COVID-19 a révélé que le volume du trafic Internet avait augmenté de 15 à 20% en une semaine, en raison de la hausse des demandes de trafic Internet des utilisateurs résidentiels, notamment pour le télétravail, les loisirs, le commerce et l'éducation [31]. En outre, une autre étude menée par Li et al. (2019) sur la blockchain a montré que les fluctuations du prix du Bitcoin ont un impact significatif sur la puissance de calcul globale du système. Un prix élevé du Bitcoin attirera plus de puissance de calcul dans le système, tandis qu'un prix bas la réduira.

Il est donc presque impossible de traiter cette énorme quantité de données en temps réel avec un ordinateur conventionnel, même si les ordinateurs actuels sont équipés de puces de processeur multicœur. Un processeur multicœur est un processeur composé de deux unités de traitement indépendantes ou plus, appelées cœurs, qui exécutent des tâches simultanément [32]. La mise en commun de centaines, de milliers, voire de millions d'unités de traitement et leur coopération pour traiter des quantités massives de données et résoudre des problèmes complexes est l'idée maîtresse du calcul haute performance [33]. Ce type d'ordinateur est généralement appelé superordinateur. Il a été au cœur des plus grandes révolutions et découvertes scientifiques des dernières décennies. Il existe trois principaux types de superordinateurs :

— Le processeur massivement parallèle, qui est un seul ordinateur composé d'un grand nombre de processeurs interconnectés par un réseau à haute vitesse. Ces processeurs sont indépendants, c'est-à-dire qu'ils ne partagent pas de mémoire, et généralement chaque processeur exécute sa propre instance d'un système d'exploitation [34]. Bien que 7,8% des 500 superordinateurs les plus puissants au monde en novembre 2021, selon le classement TOP500, soient des processeurs massivement parallèles, le superordinateur Fugaku, le premier sur la liste, est un processeur massivement parallèle. Il a atteint un score de référence de 442 Pflop/s avec 7 630 848 cœurs.
— Le cluster, qui est un ensemble d'ordinateurs autonomes interconnectés par un réseau [35]. Selon le classement TOP500, 92,2% des superordinateurs sont des clusters. Composé de 4 356 ordinateurs (pour un total de 2 414

592 cœurs) interconnectés par un Mellanox EDR Infiniband à double rail, le Summit est le deuxième superordinateur le plus puissant au monde avec une performance de 148,8 Pflop/s, soit trois fois moins puissant que le premier.

— L'ordinateur quantique, qui utilise les propriétés de la physique quantique pour effectuer des calculs. Ils visent à résoudre des problèmes extrêmement complexes que les superordinateurs les plus puissants du monde ne peuvent pas et ne pourront jamais résoudre [36]. Cette technologie a pris de l'ampleur depuis que le premier ordinateur quantique a été commercialisé par D-Wave en 2011, car depuis lors, Google, Intel, IBM et Microsoft sont à la recherche de la suprématie quantique.

I.3.2. Classifications des architectures parallèles

Les architectures parallèles sont des systèmes informatiques permettant l'exécution simultanée de multiples tâches en utilisant plusieurs processeurs. Cela peut être réalisé en juxtaposant plusieurs processeurs séquentiels ou en exécutant des instructions indépendantes de manière simultanée. La portabilité des applications parallèles reste limitée, malgré les nombreuses recherches publiées proposant des architectures et des frameworks pour améliorer cette portabilité. Il est donc crucial de bien comprendre le modèle de machine sur lequel un programme parallèle sera implémenté. Il existe divers critères de classification pour les machines parallèles, dont les plus connus sont les classifications de Flynn et de Raina.

I.3.2.1. Classification de Flynn

La classification des architectures parallèles proposée par Robert J. Flynn [37] est une taxonomie hiérarchique basée sur la gestion des flux de données et des flux d'instructions. Cette classification aide à concevoir des systèmes informatiques parallèles adaptés aux besoins spécifiques des applications et comprend quatre niveaux : SISD (Single Instruction Single Data), SIMD (Single Instruction Multiple Data), MISD (Multiple Instruction Single Data) et MIMD (Multiple Instruction Multiple Data).

SISD (Single Instruction Single Data) : Cette architecture correspond à un système informatique dans lequel un processeur unique exécute un seul flux d'instructions sur des données stockées dans une seule mémoire. Elle est typique des machines séquentielles de von Neumann.

SIMD (Single Instruction Multiple Data) : Dans cette architecture multiprocesseur, la même instruction est exécutée sur des données différentes. Ces sys-

tèmes sont appropriés pour des tâches telles que le calcul matriciel ou le traitement d'images. Cependant, leur efficacité diminue lorsqu'il s'agit de traiter des données irrégulières et fortement dépendantes de variables locales, car les processeurs peuvent rester inactifs une grande partie du temps.

MISD (Multiple Instruction Single Data) : Cette architecture traite le même flux de données en parallèle par plusieurs processeurs. Connue sous le nom de réseau systolique, elle est utilisée pour des applications spécifiques telles que le traitement d'images, le traitement de signaux et les algorithmes de cryptographie.

MIMD (Multiple Instruction Multiple Data) : Ces machines sont parfaitement parallèles, où chaque processeur exécute sa propre instruction sur ses propres données. Elles sont très flexibles et utilisées pour une large gamme d'applications informatiques, notamment les algorithmes d'apprentissage automatique, la simulation numérique et le traitement de big data.

La figure suivante résume ces quatre classes.

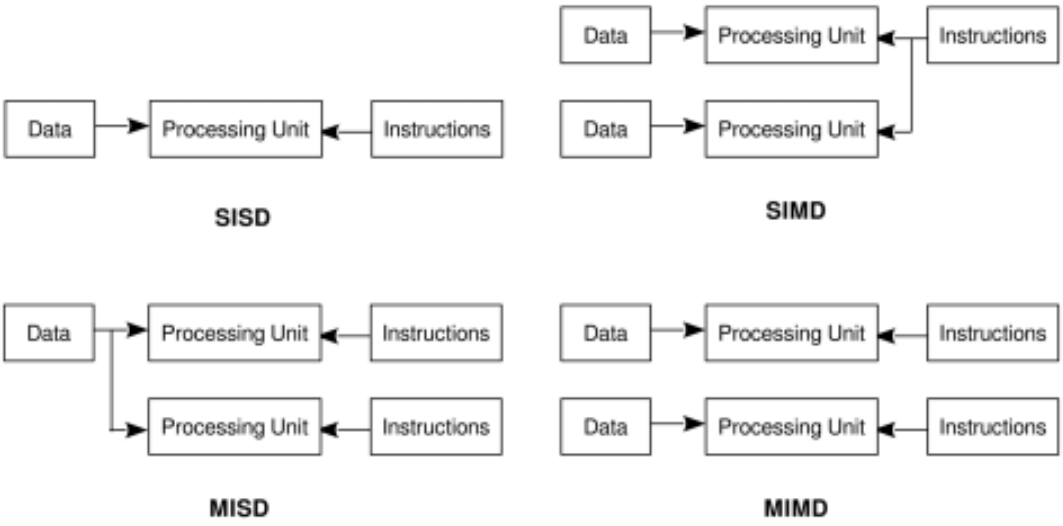


FIGURE 7 – *Classification de Flynn.[7]*

I.3.2.2. Classification de Raina

En sous-classifiant le modèle MIMD, S. K. Raina [38] propose une nouvelle classification basée sur deux critères principaux : le type d'adressage mémoire et le type d'accès mémoire .

Adressage Mémoire :

SASM (Single Address Space, Shared Memory) : Dans cette configuration, plusieurs processeurs ou cœurs partagent une même mémoire, où chaque adresse correspond à un emplacement physique unique. Cela permet à chaque processeur d'accéder directement à toutes les données sans nécessiter de transfert entre différents espaces d'adressage, facilitant une communication efficace et minimisant les problèmes de synchronisation.

DADM (Distributed Address Space, Distributed Memory) : Ici, chaque processeur ou nœud dispose de sa propre mémoire locale, et chaque adresse mémoire correspond à un emplacement physique distinct. Les processeurs n'accèdent qu'à leur mémoire locale, et toute communication inter-nœuds pour accéder à d'autres mémoires nécessite un échange de messages.

SADM (Shared Address Space, Distributed Memory) : Chaque processeur possède une mémoire locale mais peut également accéder aux mémoires locales des autres processeurs comme s'il s'agissait de sa propre mémoire. Cette configuration améliore les temps d'accès à la mémoire en réduisant la nécessité de communication inter-processus pour les données partagées.

Type d'Accès Mémoire :

NORMA (No Remote Memory Access) : Les processeurs accèdent à la mémoire locale des autres uniquement via des messages, ce qui évite la congestion du réseau et la complexité liée à l'accès distant.

UMA (Uniform Memory Access) : Dans ce modèle, l'accès à la mémoire est uniforme, avec un coût d'accès identique pour tous les processeurs.

NUMA (Non-Uniform Memory Access) : Contrairement à UMA, les performances d'accès à la mémoire dépendent de la localisation des données, ce qui est courant dans les machines parallèles car l'accès en temps constant est difficile à réaliser technologiquement.

CC-NUMA (Cache-Coherent NUMA) : Ce modèle utilise les caches locaux des processeurs pour stocker fréquemment les données utilisées, optimisant l'accès à la mémoire partagée par une politique de cohérence des caches.

OSMA (Operating System Memory Access) : Les accès aux données distantes sont gérés par le système d'exploitation, qui traite les défauts de page et les requêtes de transfert de pages.

COMA (Cache Only Memory Access) : Les mémoires locales se comportent comme des caches sans emplacement déterminé pour une donnée, facilitant une gestion flexible des données partagées.

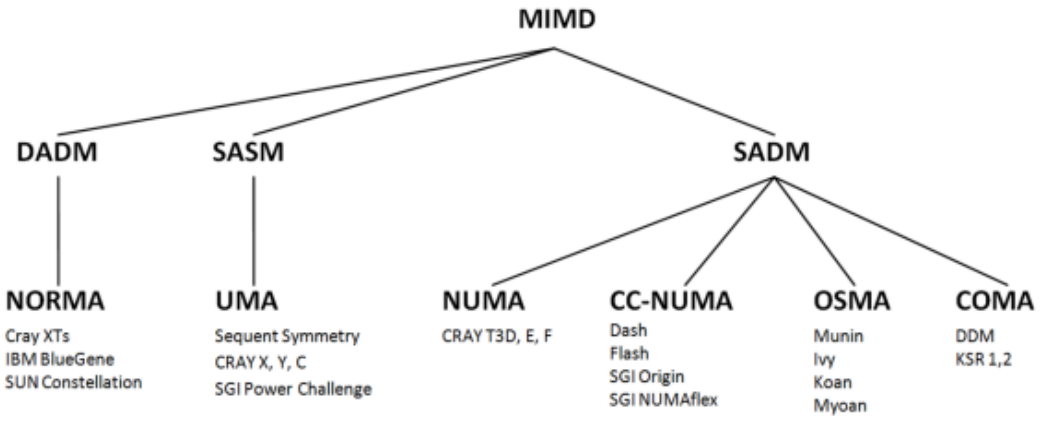


FIGURE 8 – *Classification MIMD de Reina. [8]*

I.3.2.3. Classification suivant le réseau d'interconnexion

En informatique parallèle, la topologie d'un réseau d'interconnexion désigne la manière dont les différents nœuds de calcul et les unités de mémoire sont reliés entre eux. La configuration de cette topologie a une influence majeure sur les performances d'un système parallèle, car elle affecte les temps de communication et impose des contraintes sur l'échange des données entre les nœuds. Ainsi, la topologie est un facteur crucial dans la conception d'algorithmes parallèles pour optimiser les performances globales du système [39]. Les topologies peuvent être classées en deux catégories : statiques et dynamiques.

Les topologies statiques sont prédéfinies et restent inchangées durant l'exécution du programme. Les nœuds et les liens sont arrangés de manière fixe selon une structure déterminée à l'avance. Les exemples de topologies statiques comprennent les réseaux en étoile, en anneau, en arbre, en grille et en hypercube.

À l'inverse, les topologies dynamiques peuvent être reconfigurées pendant l'exécution du programme. Les nœuds et les liens peuvent être ajoutés ou supprimés en fonction des besoins du programme. Les exemples de topologies dynamiques incluent les réseaux maillés, les réseaux toroïdaux et les réseaux de type "fat-tree". La reconfiguration de ces réseaux est rendue possible grâce à l'utilisation de commutateurs (switches) placés à des positions stratégiques dans le réseau.

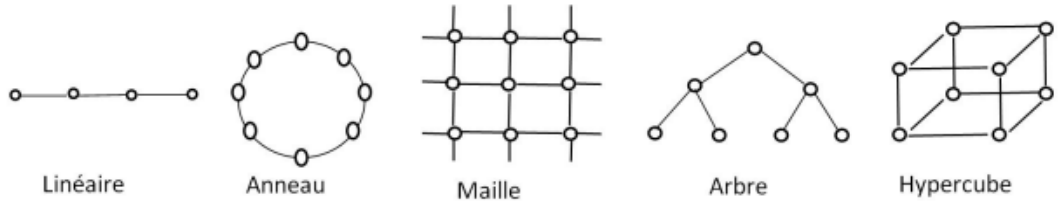


FIGURE 9 – *Réseaux d'interconnexion statique. [9]*

I.3.2.4. Classification suivant la granularité de calcul

La taille du grain d'une machine parallèle peut être définie par la longueur typique d'un morceau de code exécutable sans interruption par l'unité de calcul [39]. Les interruptions peuvent être d'origine externe, comme celles initiées par l'ordonnanceur du système d'exploitation, ou d'origine interne, comme celles déclenchées par le processus en cours de traitement pour établir une communication avec d'autres processus ou processeurs [40]. Certaines machines parallèles ne tolèrent que des interruptions internes.

On distingue trois classes de machines en fonction de la granularité de calcul :

Les machines à gros grain : Ces machines se caractérisent par un petit nombre de processeurs, généralement quelques dizaines à quelques centaines, et une mémoire partagée. Chaque processeur effectue des tâches très complexes nécessitant une grande quantité de données et des calculs intensifs, voire des applications entières. Les données sont partagées entre les processeurs, et des mécanismes de synchronisation sont souvent nécessaires pour assurer la coordination des calculs.

Les machines à grain moyen : Elles exécutent des tâches plus fines que les machines à gros grain. Chaque processeur est relativement autonome avec sa propre mémoire, et les tâches sont souvent réparties sur plusieurs processeurs.

Les machines à grain fin : Ces machines comportent un grand nombre de processeurs, souvent plusieurs milliers ou dizaines de milliers, chacun disposant de sa propre mémoire locale. Les tâches sont très fines et distribuées sur un grand nombre de processeurs, permettant un travail parallèle très efficace. Une répartition équilibrée des tâches est essentielle pour éviter les déséquilibres de charge.

I.3.3. Modèles de programmation parallèle

Il existe plusieurs modèles de programmation parallèle couramment utilisés :

— Mémoire partagée (sans threads)

— Modèle de threads
— Mémoire distribuée/passage de messages
— Parallélisme de données
— Modèle hybride
— Programme unique données multiples (SPMD)
— Programme multiple données multiples (MPMD)

Ces modèles fournissent des abstractions au-dessus des architectures matérielles et de mémoire. Même si cela peut ne pas être évident, ces modèles ne sont PAS limités à un type particulier de machine ou d'architecture mémoire. En théorie, n'importe lequel de ces modèles peut être mis en œuvre sur n'importe quel matériel sous-jacent.

Le choix du modèle à utiliser dépend souvent des ressources disponibles et des préférences personnelles. Il n'existe pas de "meilleur" modèle, bien que certains aient des implémentations plus efficaces que d'autres. Les sections suivantes présentent chacun des modèles mentionnés et examinent certaines de leurs implémentations pratiques.

I.3.3.1. Modèle de mémoire partagée (sans threads)

Ce modèle de programmation permet aux processus ou tâches de partager un espace d'adressage commun, où ils peuvent lire et écrire de manière asynchrone. Pour contrôler l'accès à la mémoire partagée et éviter les conflits, les situations de concurrence critique et les blocages, on utilise divers mécanismes tels que les verrous et les sémaphores.

Du point de vue du programmeur, un avantage de ce modèle est l'absence de "propriété" des données, ce qui facilite la communication entre les tâches. Tous les processus ont un accès égal à la mémoire partagée, ce qui simplifie le développement des programmes.

Cependant, un inconvénient majeur en termes de performances est la difficulté accrue de comprendre et de gérer la localité des données. Garder les données locales par rapport au processus qui y travaille permet de réduire les accès à la mémoire, les rafraîchissements du cache et le trafic de bus lorsque plusieurs processus utilisent les mêmes données. Néanmoins, le contrôle de la localisation des données peut être complexe et échapper au contrôle de l'utilisateur moyen.

I.3.3.2. Modèle de threads

Dans ce modèle de programmation, un unique processus, qualifié de « lourd », peut avoir plusieurs voies d'exécution simultanées, appelées « légères ».

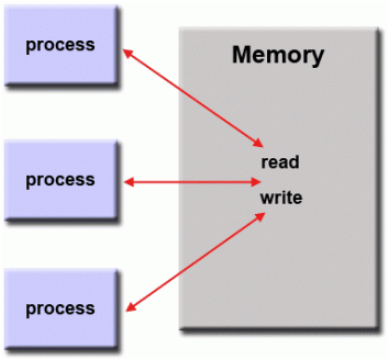


FIGURE 10 – *Modèle de mémoire partagée [10]*

Par exemple : Le programme principal, `a.out`, est conçu pour être exécuté par le système d'exploitation natif. `a.out` charge et obtient toutes les ressources système et utilisateur nécessaires pour son fonctionnement. C'est le processus « lourd ». `a.out` effectue un travail séquentiel, puis génère un certain nombre de tâches (fils d'exécution) qui peuvent être planifiées et exécutées simultanément par le système d'exploitation. Chaque fil d'exécution possède des données locales, mais partage aussi toutes les ressources de `a.out`. Cela permet d'économiser la surcharge associée à la duplication des ressources d'un programme pour chaque fil d'exécution (« léger »). Chaque fil d'exécution bénéficie également d'une vue globale de la mémoire car il partage l'espace mémoire de `a.out`. Le travail d'un fil d'exécution peut être mieux décrit comme une sous-routine au sein du programme principal. N'importe quel fil d'exécution peut exécuter n'importe quelle sous-routine en même temps que d'autres fils d'exécution. Les fils d'exécution communiquent entre eux via la mémoire globale (mise à jour des adresses). Cela nécessite des structures de synchronisation pour garantir que plusieurs fils d'exécution ne mettent pas à jour la même adresse globale en même temps. Les fils d'exécution peuvent apparaître et disparaître, mais `a.out` reste présent pour fournir les ressources partagées nécessaires jusqu'à la fin de l'application. **voir figure 11**

I.3.3.3. Modèle de mémoire distribuée/passage de messages

Ce modèle se caractérise par :

— Un groupe de tâches qui utilisent leur propre mémoire locale lors du calcul. Ces tâches peuvent être situées sur la même machine physique ou réparties sur un nombre quelconque de machines.

— Les tâches échangent des informations en envoyant et en recevant des messages.

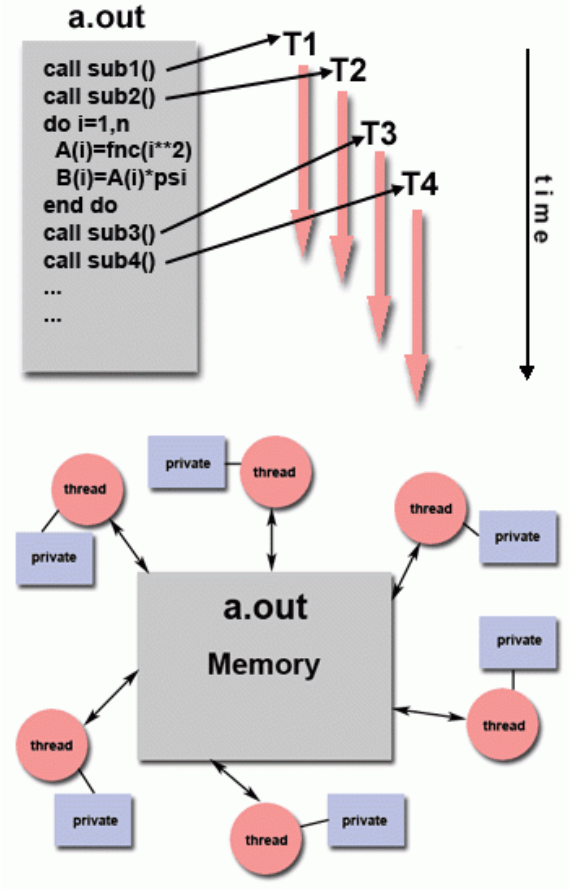


FIGURE 11 – *Modèle de threads [10]*

— Le transfert de données implique généralement la coopération de chaque processus. Par exemple, une opération d'envoi doit correspondre à une opération de réception.

Une illustration de ce modèle est donnè par la **Figure 12**

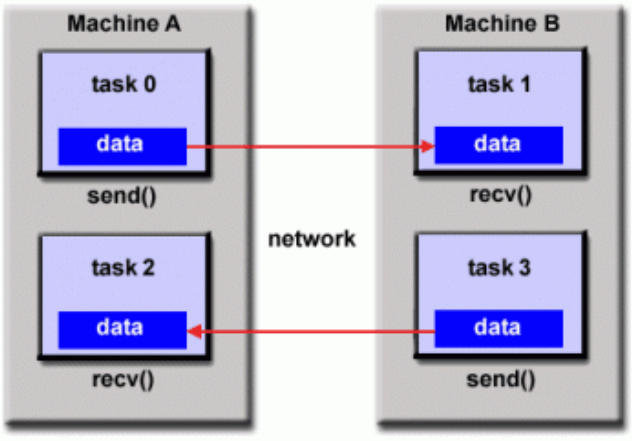


FIGURE 12 – *Modèle de mémoire distribuée [10]*

I.3.3.4. Modèle parallèle de données

Aussi connu sous le nom de modèle PGAS (Partitioned Global Address Space), il traite l'espace d'adressage globalement. La plupart des travaux parallèles se concentrent sur l'exécution d'opérations sur un ensemble de données, chaque tâche travaillant sur une partition différente. Les tâches effectuent la même opération sur leur partition, adapté aux architectures de mémoire partagée ou distribuée.

Une illustration de ce modèle est donnè par la **Figure 13**

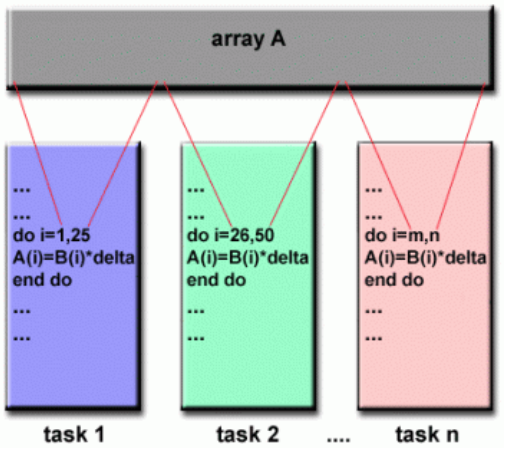


FIGURE 13 – *Modèle parallèle de données [10]*

I.3.3.5. Modèle hybride

Un modèle hybride combine plusieurs modèles de programmation, comme la combinaison de l'échange de messages (MPI) avec le modèle de fils d'exécution (OpenMP). Ce modèle convient aux clusters multicœurs, où les fils d'exécution gèrent les noyaux intensifs en calcul localement, et MPI gère la communication entre les nœuds. Un autre exemple de modèle hybride utilise MPI avec la programmation CPU-GPU, où les tâches MPI s'exécutent sur des processeurs utilisant la mémoire locale et communiquent via un réseau, tandis que les tâches intensives en calcul sont déchargées sur les GPU.

Une illustration de ce modèle est donnè par la **Figure 14**

I.3.3.6. Modèles SPMD et MPMD

— **SPMD (Programme unique données multiples)** : Toutes les tâches exécutent leur copie du même programme mais peuvent opérer sur des données différentes. Ce modèle est couramment utilisé pour les clusters multi-nœuds.

— **MPMD (Programme multiple données multiples)** : Les tâches peuvent exécuter différents programmes simultanément, adapté aux problèmes favorisant la décomposition fonctionnelle.

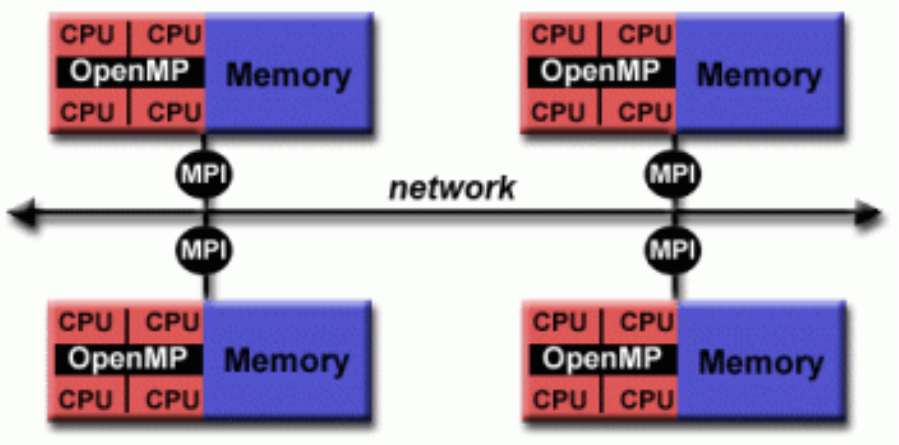


FIGURE 14 – *Modèle hybride avec MPI et OpenMP [10]*

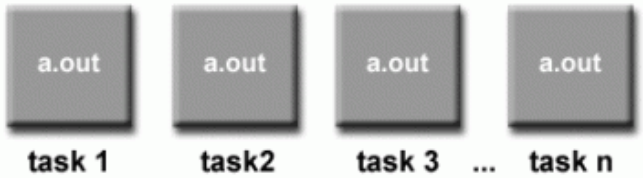


FIGURE 15 – *Modèle SPMD [10]*

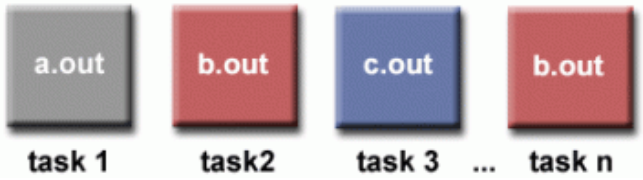


FIGURE 16 – *Modèle MPMD [10]*

I.3.4. Environnement de programmation parallèle

Une fois que l'algorithme parallèle a été conçu et ses performances théoriques évaluées, il est essentiel de procéder à son implémentation. Cette implémentation s'effectue à l'aide de frameworks de programmation parallèle. Nous discuterons ici de deux frameworks majeurs : MPI (Message Passing Interface) pour les mémoires distribuées et OpenMP (Open Multi-Processing) pour les mémoires partagées.

MPI (Message Passing Interface)

MPI est une API de haut niveau destinée à la programmation parallèle sur mémoire distribuée. Elle repose sur le paradigme de l'échange de messages et propose des interfaces en C/C++ et Fortran. Il existe plusieurs implémentations de MPI, notamment OpenMPI, MPICH, GridMPI [41] et mpi4py [42]. MPI prend en charge diverses fonctionnalités, telles que les communications unidirectionnelles, la création dynamique de processus, le multithreading et les entrées/sorties parallèles (MPI/IO).

OpenMP (Open Multi-Processing)

OpenMP est une interface de programmation conçue pour le calcul parallèle sur des architectures à mémoire partagée [43]. Cette API est supportée sur de nombreuses plateformes, y compris GNU/Linux, OS X et Windows, pour les langages de programmation C, C++ et Fortran. OpenMP se présente sous la forme d'un ensemble de directives de compilation, de bibliothèques logicielles et de variables d'environnement, fournissant un cadre pour créer des threads, synchroniser les opérations et gérer la mémoire partagée.

Les programmes OpenMP sont compilés en programmes multithreads où les threads partagent le même espace d'adressage mémoire, rendant ainsi les communications inter-threads très efficaces. Comparé à l'utilisation de pthreads et à l'emploi de mutex et de variables de condition, OpenMP simplifie grandement la programmation multithread en transformant automatiquement le code séquentiel en code parallèle selon les directives spécifiées [41]. Cela permet aux développeurs de créer des programmes multithreads sans une compréhension approfondie des mécanismes de multithreading.

Le runtime d'OpenMP maintient le pool de threads et fournit un ensemble de bibliothèques. Lorsqu'un bloc parallèle est initié, un seul thread de contrôle (maître) se divise en plusieurs threads (esclaves), et un nouveau thread séquentiel démarre une fois que tous les threads divisés ont terminé. OpenMP est à la fois portable et extensible, permettant un développement rapide d'applications parallèles tout en restant proche du code séquentiel.

Joblib

Joblib est une bibliothèque optimisée pour la parallélisation de boucles et la gestion de la mémoire en Python. Elle est particulièrement utilisée dans le cadre de l'apprentissage automatique pour paralléliser des tâches telles que l'entraînement de modèles. Joblib se distingue par sa capacité à sérialiser des objets complexes Python, ce qui facilite le partage de données entre les processus parallèles sans nuire aux performances. Elle permet également de mettre en cache les résultats de calculs fréquents, ce qui peut considérablement améliorer l'efficacité des workflows répétitifs. En outre, Joblib offre une interface simple et intuitive pour la parallélisation, utilisant des décorateurs et des gestionnaires de contexte pour préciser quelles parties du code doivent être exécutées en parallèle.

Concurrent.futures

La bibliothèque `concurrent.futures` offre une interface de haut niveau pour exécuter des appels asynchrones. Elle permet de gérer des pools de threads ou de processus, facilitant ainsi la parallélisation de tâches indépendantes. Avec `concurrent.futures`, les développeurs peuvent utiliser deux types d'exécuteurs : **ThreadPoolExecutor** et `ProcessPoolExecutor`. `ThreadPoolExecutor` est idéal pour les tâches liées à l'I/O où les threads peuvent attendre des opérations d'entrée/sortie, tandis que `ProcessPoolExecutor` est mieux adapté pour les tâches CPU-bound, où les processus Vous pouvez utiliser pleinement les cœurs du processeur. La bibliothèque simplifie également la gestion des futures, permettant de vérifier l'état des tâches, d'attendre leur achèvement ou de récupérer les résultats de manière non bloquante.

Multitraitement

La bibliothèque `multiprocessing` de Python permet de créer des processus parallèles qui s'exécutent sur plusieurs cœurs de CPU. Contrairement aux threads, les processus créés par `multiprocessing` ne partagent pas le même espace mémoire, ce qui génère les problèmes de contention des threads mais nécessitent des mécanismes de communication inter-processus, tels que les pipes et les fichiers de messages. `multiprocessing` propose également des outils de synchronisation tels que les verrous, les sémaphores, les barrières et les événements pour coordonner les activités des processus parallèles. En outre, cette bibliothèque inclut un support pour le partage de données via des objets partagés et des serveurs de gestion de mémoire partagée, facilitant la communication entre les processus. Cela en fait une solution robuste pour les applications nécessitant une parallélisation intensive des tâches CPU-bound.

Threading

La bibliothèque `threading` de Python permet de créer des programmes multithreads. Bien que les threads partagent le même espace mémoire, le Global Interpreter Lock (GIL) de Python peut limiter les performances en parallèle pur, car il permet à un seul thread Python d'exécuter le bytecode à la fois. Cependant, `threading` reste extrêmement utile pour les opérations I/O liées, où les threads peuvent attendre des opérations d'entrée/sortie sans être bloqués par le GIL. La bibliothèque fournit des abstractions de haut niveau pour la création et la gestion des threads, incluant des classes et des méthodes pour démarrer, arrêter et joindre des threads, ainsi que des mécanismes de synchronisation tels que les verrous (locks),

les conditions (conditions) et les sémaphores (sémaphores). Ces outils permettent aux développeurs de concevoir des programmes multithreads complexes et de gérer efficacement la concurrence et les accès aux ressources partagées.

I.3.5. Défis de la conception d'algorithmes parallèles

La conception d'algorithmes parallèles présente plusieurs défis importants qui doivent être surmontés pour exploiter pleinement les avantages des architectures de calcul parallèle. Ces défis incluent l'équilibrage de la charge, la communication et la synchronisation, la tolérance aux pannes, et la gestion des ressources.

— **Équilibrage de la charge** : L'équilibrage de la charge consiste à répartir les tâches de manière uniforme entre les processeurs afin d'éviter que certains processeurs soient surchargés tandis que d'autres restent inactifs. Un mauvais équilibrage de la charge peut entraîner des temps d'attente inutiles et réduire l'efficacité globale du système. Les techniques d'équilibrage de la charge incluent le partitionnement statique, où les tâches sont distribuées avant l'exécution, et le partitionnement dynamique, où les tâches sont redistribuées en fonction de la charge de travail actuelle.

— **Communication et synchronisation** : Les processeurs parallèles doivent souvent échanger des données pour accomplir leur tâche. La communication entre les processeurs peut introduire des latences significatives, surtout dans les systèmes à mémoire distribuée. Minimiser le temps de communication est crucial pour la performance des algorithmes parallèles. De plus, la synchronisation entre les processeurs est essentielle pour assurer la cohérence des données et éviter les conflits d'accès. Les primitives de synchronisation comme les barrières et les verrous sont couramment utilisées, mais elles peuvent introduire des goulots d'étranglement si elles ne sont pas gérées correctement.

— **Tolérance aux pannes** : Dans les systèmes parallèles, la défaillance d'un seul processeur peut compromettre l'exécution de l'ensemble de l'application. La tolérance aux pannes implique la capacité de détecter les défaillances et de récupérer sans perdre de données ou de temps de calcul de manière significative. Les techniques de tolérance aux pannes incluent la réplication des tâches, les points de contrôle périodiques (checkpointing) et la redondance des calculs.

— **Gestion des ressources** : La gestion efficace des ressources, telles que les processeurs, la mémoire et la bande passante réseau, est essentielle pour

optimiser les performances des algorithmes parallèles. Cela inclut la gestion dynamique des ressources en fonction des besoins changeants de l'application et l'allocation efficace des ressources pour éviter les conflits et les surutilisations. Des stratégies de gestion des ressources bien conçues peuvent aider à maximiser l'utilisation des ressources disponibles et à améliorer l'évolutivité des algorithmes.

— **Débogage et validation** : Le débogage et la validation des algorithmes parallèles sont plus complexes que pour les algorithmes séquentiels en raison de la nature concurrente de leur exécution. Les erreurs de synchronisation, les conditions de course et les blocages sont des problèmes courants qui peuvent être difficiles à détecter et à reproduire. Les outils de débogage spécialisés pour les systèmes parallèles, tels que les analyseurs de performances et les débogueurs distribués, sont indispensables pour identifier et corriger ces problèmes.

— **Évolutivité** : L'évolutivité des algorithmes parallèles est la capacité de maintenir ou d'améliorer les performances lorsque le nombre de processeurs augmente. Un algorithme parallèle évolutif doit minimiser les coûts de communication et de synchronisation, et assurer un bon équilibrage de la charge à grande échelle. Les algorithmes qui ne sont pas conçus pour être évolutifs peuvent voir leurs performances se dégrader lorsqu'ils sont exécutés sur un grand nombre de processeurs.

I.4. Conclusion

La fouille de données et la parallélisation sont essentielles en informatique moderne pour analyser de grandes quantités de données et résoudre des problèmes complexes. La fouille de données extrait des motifs cachés, tandis que la parallélisation améliore les performances en exploitant les architectures de calcul modernes.

Ces techniques sont cruciales dans divers secteurs, mais posent des défis en matière de qualité des données, d'éthique et de complexité computationnelle. La parallélisation nécessite une gestion rigoureuse de la charge et des ressources.

En résumé, la fouille de données et la parallélisation sont vitales pour exploiter les technologies actuelles, permettant d'analyser efficacement des données massives et d'ouvrir la voie à de nouvelles découvertes.

II

CHAPITRE

GÉNÉRALITÉS SUR LES MODÈLES MULTI-TEMPORELS ET ÉTAT DE L'ART

SOMMAIRE

II.1 - Introduction 35
II.2 - Généralités sur les modèles multi-temporels 36
II.3 - État de l'art 43
II.4 - Limites de recherche 53
II.5 - Conclusion 54

II.1. Introduction

L'étude des modèles multi-temporels est une branche cruciale de la fouille de données, particulièrement pertinente pour les applications nécessitant l'analyse des données séquentielles. Avec l'explosion des volumes de données générés par des capteurs, des transactions financières, des systèmes de surveillance, et d'autres sources, la compréhension et l'exploitation des relations temporelles deviennent essentielles. Les modèles multi-temporels permettent de capturer ces relations, offrant des moyens d'extraire des motifs significatifs et de prédire des tendances futures.

La fouille de données temporelles inclut diverses techniques et algorithmes pour analyser des séquences de données afin de découvrir des motifs récurrents, des anomalies, et des relations causales. Ces modèles trouvent des applications dans de nombreux domaines tels que la finance, la biologie, la santé, le commerce électronique, et la surveillance environnementale.

Ce chapitre présente un état de l'art desmodèles multi-temporels, en explorant diverses approches et algorithmes développés dans ce domaine. Nous examinerons

les contributions majeures et les méthodologies adoptées par différents chercheurs pour extraire et analyser les motifs temporels. Ces approches varient des règles d'association aux modèles multi-temporels, en passant par les modèles d'association temporelle pendant. Nous discuterons également des méthodes de découverte des motifs multi-temporels et leur application aux bases de données financières.

II.2. Généralités sur les modèles multi-temporels

La fouille de données temporelles est un domaine crucial qui exploite les aspects temporels des données pour découvrir des motifs, des tendances, et des régularités au fil du temps. Cette discipline s'étend au-delà de la simple analyse statistique de séries temporelles pour inclure des méthodologies sophistiquées qui intègrent l'ordre temporel des événements afin de fournir des insights plus profonds et stratégiques. Les modèles multi-temporels permettent aux chercheurs et professionnels de plusieurs domaines, tels que la finance, la santé, le marketing et la sécurité publique, de prévoir des événements futurs, de comprendre les comportements complexes, et de planifier des actions basées sur des prédictions fiables.

II.2.1. Cadre formel et définition des modèles multi-temporels

Les modèles multi-temporels en fouille de données sont cruciaux pour identifier des relations entre des séquences d'événements ordonnées dans le temps. Ces modèles sont largement exploités dans des domaines où l'analyse temporelle est essentielle, tels que les marchés financiers, la surveillance de la santé ou la prévision météorologique. La fouille de motifs multi-temporels implique la découverte d'associations entre des événements qui se produisent à différents moments mais sont liés par des contraintes temporelles. Pour illustrer ces concepts, nous définirons les termes essentiels et présenterons la structure formelle utilisée dans les bases de données temporelles :

Définition des états et des événements : Les objets réels (tels que les personnes, machines, et plantes) peuvent être représentés par des attributs. La valeur d'un attribut pour un objet représente l'état de cet objet, formulé par le triplet $s = (o, a, v)$, où o est l'objet, a un attribut de o, et v la valeur de a.

Exemple 1 : $s =$ (stock A, price, increase) décrit l'état où le prix du stock A augmente.

Ces états se produisent durant des intervalles spécifiques, représentés par des événements $e = (s, st, et)$, où st et et dénotent respectivement le début et la fin de

l'état. Ainsi, $e = (s, 1, 3)$ signifie que l'état s se produit de $t = 1$ à $t = 3$.

Structure d'une base de données temporelle : Considérons S comme un ensemble de n états $S = \{s_1, s_2, \ldots, s_n\}$. Chaque s_i peut correspondre à plusieurs événements e_{ij}, où j varie. La base de données temporelle, D_T, est alors représentée par un ensemble d'événements triés par leur début.

Définition 1 : *Pour un événement e_A avec l'intervalle (st_A, et_A), et une fenêtre temporelle prédéfinie, $C_A = [st_A, et_A + window]$ définit la contrainte de fenêtre pour e_A. Si un autre événement e_B se trouve dans C_A, alors e_B satisfait la contrainte de fenêtre de e_A.*

La contrainte de fenêtre, utilisée comme mesure d'intérêt, indique que si deux événements sont temporellement éloignés, ils ne sont pas considérés comme associés sémantiquement. Cette mesure est particulièrement utile dans divers contextes analytiques.

Event	State	st	et	Event	State	st	et
e_{11}	s_1	1	5	e_{32}	s_3	33	38
e_{31}	s_3	1	4	e_{62}	s_6	34	39
e_{51}	s_5	2	10	e_{42}	s_4	25	37
e_{21}	s_2	2	8	e_{52}	s_5	27	42
e_{61}	s_6	2	5	e_{23}	s_2	28	32
e_{22}	s_2	4	6	e_{12}	s_1	30	40
e_{41}	s_4	3	7	e_{33}	s_3	30	38
...	...	...	...	...	...	...	...

FIGURE 17 – *Exemple d'une base de données temporelle [1]*

Définition de l'instance temporelle : Une instance temporelle entre deux événements e_A et e_B est représentée par $\phi_1 := e_A \xrightarrow{R} e_B$, où R est un prédicat temporel spécifiant la nature de la relation temporelle entre les événements, et $R \in \{E, B, D, O\}$ correspondant respectivement à égal, avant, pendant, et chevauchant.

Définition d'un motif temporel : Un motif est défini par une relation temporelle entre états, par exemple $S_i \xrightarrow{R} S_j$, où R est un prédicat temporel indiquant la nature de la relation entre ces états dans une séquence de données. Un motif est

considéré comme temporel s'il est soutenu par une instance temporelle significative.

Exemple 2 : le motif temporel $S_1 \xrightarrow{O} S_2$ est soutenu par les instances temporelles $e_{11} \xrightarrow{O} e_{21}$, $e_{11} \xrightarrow{O} e_{22}$, et $e_{12} \xrightarrow{O} e_{23}$ sur la Figure 7.

Définition 2 : Un motif temporel est constitué d'un ensemble de n états, noté $S = \{s_1, s_2, \ldots, s_n\}$, et d'un ensemble de prédicats temporels $\{D, B, O, E\}$, qui définissent les relations entre ces états. La longueur du motif est déterminée par le nombre de prédicats k, et le motif lui-même est dénoté par Φ_k. Les différents types de motifs sont définis comme suit :

1. *Lorsque* $k = 0$, $\Phi_0 := s_0$, $s_0 \in S$ *(que nous appelons un motif temporel dégénéré) ;*
2. *Lorsque* $k = 1$, $\Phi_1 := (\Phi_0 \xrightarrow{R_1} s_1) : (s_0 \xrightarrow{R_1} s_1)$, $R_1 \in \{D, B, O, E\}$ *;*
3. *Lorsque* $k = 2$, $\Phi_2 := (\Phi_1 \xrightarrow{R_2} s_2) : (s_0 \xrightarrow{R_1} s_1 \xrightarrow{R_2} s_2)$, $R_1, R_2 \in \{D, B, O, E\}$ *;*
4. *Lorsque* $k > 2$, $\Phi_k := (\Phi_{k-1} \xrightarrow{R_k} s_k) : (s_0 \xrightarrow{R_1} s_1 \xrightarrow{R_2} \ldots \xrightarrow{R_k} s_k)$, $R_j \in \{D, B, O, E\}$ *où* $j = 1, 2, \ldots, k$.

.

Exemple 3 (pour $k = 0$) :

— **Lorsque** $k = 0$, le motif contient seulement un état. Par exemple, "l'augmentation du prix de l'action A" est un motif dégénéré temporel car il n'implique aucune relation temporelle avec un autre état.

— **Lorsque** $k = 1$, cela signifie que deux états ont une relation temporelle. Par exemple, "l'augmentation du prix de l'action A" $\xrightarrow{B}$ "l'augmentation de l'indice boursier de Shanghai" indique que l'augmentation du prix de l'action A se produit avant l'augmentation de l'indice boursier.

— **Lorsque** $k = 2$, il représente plusieurs états ayant des relations temporelles séquentielles. Par exemple, "l'augmentation du prix de l'action A" $\xrightarrow{B}$ "l'augmentation de l'indice boursier de Shanghai" $\xrightarrow{B}$ "l'augmentation de l'indice boursier de Shenzhen" illustre une chaîne de relations où les événements se suivent dans un ordre spécifique.

— **Pour** $k > 2$, un motif temporel complexe peut impliquer plusieurs relations temporelles entre plus de trois états. Par exemple, "la baisse du prix de l'action B" $\xrightarrow{D}$ "l'augmentation du prix de l'action A" $\xrightarrow{B}$ "l'augmentation de l'indice boursier de Shanghai" représente un motif temporel multi-temporel

avec deux prédicats différents D et B, montrant comment différents événements sont interconnectés à travers le temps.

.

Définition 3 : Étant donné un motif temporel ϕ, noté sous la forme $\phi = S_0 \xrightarrow{R_1} S_1 \xrightarrow{R_2} S_2 \rightarrow \cdots \xrightarrow{R_k} S_k$, où chaque R_j est un prédicat temporel parmi $\{D, B, O, E\}$ et $j = 1, 2, \ldots, k$, avec une instance temporelle φ qui supporte ϕ, l'intervalle de temps effectif pour ϕ supportant φ, noté t_ϕ, est défini comme suit :

$$t_\phi = t_{\phi_k} \cup t_{\phi_{k-1}} \cup \cdots \cup t_{\phi_1} \cup t_{\phi_0},$$

où $t_{\phi_j} = (st_{\phi_j}, et_{\phi_j})$ est l'intervalle de temps de l'événement e_{ϕ_j}, et $j = 0, 1, 2, \ldots, k$. L'intervalle t_ϕ est défini comme $t_\phi = (st_\phi, et_\phi)$, où $st_\phi = \min\{st_{\phi_j} | j = 0, 2, \ldots, k\}$ et $et_\phi = \max\{et_{\phi_j} | j = 0, 2, \ldots, k\}$. L'ensemble des intervalles de temps effectifs pour toutes les instances temporelles supportant ϕ est appelé l'ensemble des intervalles de temps effectifs pour le motif temporel ϕ.

Exemple 4 : Pour le motif temporel $\phi = S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2$ supporté par l'instance temporelle $\varphi = e_{31} \xrightarrow{D} e_{11} \xrightarrow{D} e_{21}$, l'intervalle de temps effectif pour ϕ supportant φ est $t_\phi = (1, 8)$, calculé comme $(\min\{1, 1, 2\}, \max\{4, 5, 8\}) = (1, 8)$.

Définition 4 : Dans une base de données temporelle, pour un motif temporel donné ϕ, le nombre d'instances temporelles qui supportent ce motif est noté comme $\text{supp}(\phi)$. Le degré de support d'un motif temporel ϕ est alors défini par :

$$\text{support}(\phi) = \frac{\text{supp}(\phi)}{|E|},$$

où $|E|$ est le maximum des cardinalités des ensembles d'événements E_j qui supportent chaque état du motif dans la base de données temporelle D_T, c'est-à-dire $|E| = \max(\{|E_j|\}_{j=1}^{N})$ et N est le nombre d'états dans la base de données D_T. Chaque E_j est l'ensemble des événements qui supportent l'état s_j, et e_{ij} est un événement qui supporte l'état s_j, c'est-à-dire $e_{ij} \in E_j$.

Exemple 5 : Pour le motif temporel $\phi = S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2$ dans la Table 1, supporté par les instances temporelles $e_{31} \xrightarrow{D} e_{11} \xrightarrow{D} e_{21}$ et $e_{33} \xrightarrow{D} e_{12} \xrightarrow{D} e_{23}$, avec $E_1 = \{e_{11}, e_{12}, e_{13}\}, E_2 = \{e_{21}, e_{22}, e_{23}\}, E_3 = \{e_{31}, e_{32}, e_{33}\}, E_4 = \{e_{41}, e_{42}\}, E_5 = \{e_{51}, e_{52}\}, E_6 = \{e_{61}, e_{62}\}$ et ainsi $|E| = \max(3, 3, 3, 2, 2, 2) = 3$. Le support de ϕ est donc $\text{support}(S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2) = 2/3$. Le degré de support mesure la fréquence (force) d'un motif temporel dans une base de données temporelle donnée.

Définition 5 : Dans une base de données temporelle, pour un motif temporel ϕ comprenant k prédicats temporels et connecté à $k+1$ états, soit $\phi = S_0 \xrightarrow{R_1} S_1 \xrightarrow{R_2} S_2 \ldots \xrightarrow{R_k} S_k$, la confiance d'un motif temporel ϕ est définie comme suit :

$$\text{confidence}(\phi) = \frac{\text{supp}(\phi_k) \times \text{confidence}(\phi_{k-1})}{\text{supp}(\phi_{k-1})}, \text{ pour } R_k \neq D$$

$$\text{confidence}(\phi) = \frac{\text{supp}(\phi)}{\text{supp}(\phi_k)} \times \text{confidence}(S_k), \text{ pour } R_k = D$$

où ϕ_k est le motif jusqu'à l'état S_k, et confidence(S_k) est 1 si ϕ est un motif dégénéré.

Exemple 6 : sur la figure 7, pour $\phi = S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2$ avec $\text{supp}(S_3 \xrightarrow{D} S_1) = 2$ et $\text{supp}(S_1 \xrightarrow{D} S_2) = 2$, la confiance est calculée comme suit :

$$\text{confidence}(S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2) = \frac{2}{2} \times 100\% = 100\%$$

Ce calcul illustre que l'état S_3 précédera S_1 et S_2 à 100% des occasions dans cette base de données temporelle.

Les définitions du degré de support et de la confiance d'un motif temporel sont cruciales pour évaluer l'intérêt et la fréquence d'un motif dans une base de données temporelle. Un motif est considéré comme fréquent ou qualifié si son support et sa confiance dépassent les seuils minimums prédéfinis α et β, respectivement.

Définition 6 : Étant donné un motif temporel $\phi = S_0 \xrightarrow{R_1} S_1 \xrightarrow{R_2} S_2 \ldots \xrightarrow{R_k} S_k$, un sous-motif $\phi' = S_{j1} \xrightarrow{R_{j1}} S_{j2} \xrightarrow{R_{j2}} \ldots \xrightarrow{R_{jm}} S_{jm}$ est défini, où $0 \leq j1 \leq j2 \ldots \leq jm \leq k$ et $0 \leq m \leq k$. Les indices $j1, j2, \ldots, jm$ sont des nombres naturels séquentiels, c'est-à-dire $j_p = j_{p-1} + 1, p = 1, 2, \ldots, m$ et $j1 = 0$, alors ϕ' est appelé un sous-motif séquentiel de ϕ.

Définition 7 : Pour un motif temporel ϕ et un sous-motif ϕ', pour toute instance ϕ de ϕ avec son intervalle de temps effectif $t_\phi \in T(\phi)$, s'il existe une instance ϕ' de ϕ' avec son intervalle de temps effectif $t_{\phi'} \in T(\phi')$ tel que $t_{\phi'} \subseteq t_\phi$, alors nous appelons ϕ' un sous-motif supporté de ϕ.

Exemple 7 : sur la figure 7, pour le motif temporel $\phi = S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2$, où $S_3 \xrightarrow{D} S_1$ et $S_1 \xrightarrow{D} S_2$ sont ses sous-motifs séquentiels. De plus, dans la Table 1, le motif temporel $\phi = S_3 \xrightarrow{D} S_1 \xrightarrow{D} S_2$ est supporté par les instances temporelles $\phi_1 = e_{31} \xrightarrow{D} e_{11} \xrightarrow{D} e_{21}$ et $\phi_2 = e_{33} \xrightarrow{D} e_{12} \xrightarrow{D} e_{23}$ avec des intervalles de temps effectifs

$t_{\phi_1} = (1,8)$ et $t_{\phi_2} = (28,40)$, respectivement. Maintenant, considérons un sous-motif $\phi' = S_3 \xrightarrow{D} S_2$, il a ses instances $\phi'_1 = e_{31} \xrightarrow{D} e_{21}$ et $\phi'_2 = e_{33} \xrightarrow{D} e_{23}$ avec des intervalles de temps effectifs $t_{\phi'_1} = (1,8)$ et $t_{\phi'_2} = (28,38)$, respectivement. Puisque $(1,8) \subseteq (1,8)$ et $(28,38) \subseteq (28,40)$, alors ϕ' est un sous-motif supporté de ϕ.

II.2.2. L'intérêt de l'extraction des modèles multi-temporels

L'extraction des modèles multi-temporels dans les bases de données est devenue une discipline pivot pour les entreprises et les institutions qui souhaitent exploiter pleinement les potentialités cachées dans les données séquentielles. Cette technique apporte une valeur inestimable pour une variété d'applications, allant de la prévision économique à la surveillance de la santé en passant par la gestion de l'énergie.

Prévisions Améliorées : L'analyse des modèles multi-temporels permet aux organisations de prédire les comportements futurs avec une précision accrue. En finance, par exemple, cela aide les analystes à anticiper les fluctuations du marché, réduisant ainsi les risques et maximisant les rendements des investissements. Dans le secteur météorologique, cela permet de prédire des phénomènes extrêmes avec une plus grande anticipation, sauvant potentiellement des vies et minimisant les dommages économiques.

Personnalisation des Services : Dans le domaine de la santé, les modèles multi-temporels aident à personnaliser les traitements pour les patients en analysant les tendances des symptômes et des réactions aux médicaments sur de longues périodes. Ceci conduit à des soins plus efficaces et plus personnalisés, améliorant les résultats pour les patients tout en optimisant les ressources hospitalières.

Optimisation des Opérations : Dans les industries manufacturières et logistiques, l'extraction de modèles multi-temporels permet une gestion plus précise des stocks et une meilleure planification des ressources. Cela réduit les coûts, augmente l'efficacité, et améliore le service client en prévoyant la demande et en ajustant automatiquement l'offre.

II.2.3. Défis de l'extraction des modèles multi-temporels

Bien que l'extraction des modèles multi-temporels offre de nombreux avantages, elle pose également un certain nombre de défis substantiels qui doivent être surmontés pour réaliser son plein potentiel.

Volume et Variété des Données : Le volume croissant des données générées par les dispositifs numériques et les capteurs pose un défi significatif. Le traitement et l'analyse de grands volumes de données temporelles exigent des capacités de

stockage et de calcul importantes, ce qui nécessite des investissements significatifs en infrastructure technologique.

Dynamique des modèles multi-temporels : Les modèles multi-temporels ne sont pas statiques ; ils évoluent avec le temps. Adapter continuellement les modèles pour qu'ils restent pertinents face à des données en constante évolution est une tâche complexe qui nécessite des algorithmes avancés et souvent une intervention humaine pour ajuster et valider les modèles.

Problèmes de Confidentialité et de Sécurité : L'extraction de données temporelles implique souvent des informations sensibles et personnelles. Assurer la protection de ces données contre les accès non autorisés et les fuites tout en respectant les réglementations strictes sur la confidentialité est un aspect crucial qui doit être rigoureusement géré.

Compétences et Ressources : Le manque de compétences spécialisées en analyse de données temporelles peut limiter la capacité des organisations à mettre en œuvre des solutions efficaces. Former ou recruter des talents qualifiés représente un investissement important pour les entreprises souhaitant capitaliser sur les techniques d'analyse temporelle.

II.3. État de l'art

II.3.1. L'algorithme Apriori

II.3.1.1. Contexte et origine

L'algorithme Apriori, développé par Rakesh Agrawal et Ramakrishnan Srikant en 1994, a marqué un tournant dans le domaine de l'extraction des règles d'association [22]. Cet algorithme est conçu pour identifier des combinaisons fréquentes d'articles dans des transactions de clients. Son application est vaste et inclut des domaines tels que le commerce de détail, le marketing et la gestion des stocks. Grâce à Apriori, les entreprises peuvent découvrir des relations cachées entre les produits, optimisant ainsi leurs stratégies commerciales et logistiques.

II.3.1.2. Principe de fonctionnement

L'algorithme Apriori repose sur deux étapes principales : la génération des itemsets fréquents et la génération des règles d'association.

1. **Génération des itemsets fréquents :**
   — L'algorithme commence par identifier tous les itemsets (ensembles d'articles) fréquents dans la base de données. Il utilise une approche itérative basée sur le principe d'anti-monotonie, qui stipule qu'un itemset de taille k est fréquent uniquement si tous ses sous-ensembles de taille $k-1$ sont également fréquents [22]. Cela signifie que pour qu'un itemset de trois articles soit fréquent, tous les itemsets de deux articles inclus dans celui-ci doivent également être fréquents.
2. **Génération des règles d'association :**
   — À partir des itemsets fréquents, l'algorithme génère des règles d'association et les évalue en termes de support et de confiance pour déterminer leur pertinence. Par exemple, une règle d'association pourrait être de la forme $\{\text{pain}\} \rightarrow \{\text{beurre}\}$, indiquant que si un client achète du pain, il est probable qu'il achète également du beurre.

II.3.1.3. Pseudocode de l'algorithme Apriori

L'algorithme suit les étapes suivantes pour générer les itemsets fréquents et les règles d'association :

Entrées: Une base de données D contenant des transactions, un seuil de support minimal minsup
Sortie : L'ensemble des itemsets fréquents
début
  $F_1 = \{\text{items fréquents de taille } 1\}$;
  $k = 1$;
  while $F_k \neq \emptyset$ **do**
    $C_{k+1} = \text{apriori-gen}(F_k)$ // Nouveaux candidats générés à partir de F_k
    for *toutes les transactions t dans la base de données D* **do**
      $C'_t = \text{subset}(C_{k+1}, t)$ // Candidats contenus dans t
      for *chaque candidat c dans* C'_t **do**
        $c.\text{count}++$ // Incrémenter le compteur de tous les candidats dans C_{k+1} contenus dans t
      endfor
    endfor
    $F_{k+1} = \{c \in C_{k+1} \mid c.\text{count} \geq \text{support minimum}\}$ // Candidats dans C_{k+1} avec le support minimum
    $k = k + 1$;
  end
  return $\bigcup_k F_k$;
fin

Algorithm 1: Algorithme Apriori

II.3.1.4. Explication détaillée avec exemple

Pour illustrer le fonctionnement de l'algorithme Apriori, nous allons décrire les différentes étapes et concepts clés en détail, en utilisant un exemple de transactions dans un supermarché :

Clients	Liste d'items
1	I1, I2, I3
2	I2, I4
3	I2, I3
4	I1, I2, I4
5	I1, I3
6	I2, I3
7	I1, I3
8	I1, I2, I3, I5
9	I1, I2, I3

TABLE III – *Exemple de transactions dans un supermarché*

1. **Détermination des 1-itemsets fréquents :**

— L'algorithme commence par examiner tous les items individuels dans la base de données et calcule leur fréquence d'apparition. Seuls les items dont la fréquence dépasse le seuil de support minimal (minsup) sont conservés. Par exemple, si un item apparaît dans au moins 22.22% des transactions, il est considéré comme fréquent.

— **Calcul des fréquences :**

1-itemset	Fréquence/9
I1	6
I2	7
I3	6
I4	2
I5	1

TABLE IV – *1-itemsets fréquents*

2. **Formation des 2-itemsets :**
   — À partir des 1-itemsets fréquents, l'algorithme génère tous les possibles 2-itemsets (combinaisons de deux items). Il calcule ensuite la fréquence de chaque 2-itemset et conserve ceux qui sont fréquents selon le seuil de support minimal.
   — **Calcul des fréquences :**

2-itemset	Fréquence/9
I1, I2	4
I1, I3	4
I2, I3	4
I2, I4	2

TABLE V – *2-itemsets fréquents*

3. **Détermination des 3-itemsets fréquents :**
   — De manière similaire, l'algorithme forme des 3-itemsets à partir des 2-itemsets fréquents et filtre ceux qui répondent au critère de fréquence.
   — **Calcul des fréquences :**

3-itemset	Fréquence/9
I1, I2, I3	2

TABLE VI – *3-itemsets fréquents*

4. **Calcul des supports et confiances :**
   — Pour chaque itemset fréquent, l'algorithme génère des règles d'association et les évalue en termes de support et de confiance.

— Le **support** d'une règle $X \Rightarrow Y$ est calculé comme le ratio du nombre de transactions contenant $X \cup Y$ sur le nombre total de transactions, soit $\frac{\text{freq}(X,Y)}{N}$.

— La **confiance** d'une règle $X \Rightarrow Y$ est déterminée par le ratio du support de $X \cup Y$ sur le support de X, soit $\frac{\text{freq}(X,Y)}{\text{freq}(X)}$. Pour assurer que les règles générées sont pertinentes, seules celles dont la confiance dépasse un certain seuil (par exemple, 50%) sont retenues.

5. **Formation des règles d'association :** À partir des itemsets fréquents, nous générons des règles d'association en suivant les étapes ci-dessous :
   — Pour chaque itemset fréquent I, identifier tous les sous-ensembles non vides.
   — Pour chaque sous-ensemble S, créer une règle de la forme $S \rightarrow (I - S)$.
   — Calculer la confiance de la règle comme le ratio du support de I sur le support de S.
   — Retenir les règles dont la confiance est supérieure ou égale au seuil de confiance prédéfini.
   — La règle $\{1\} \rightarrow \{2, 3\}$ a une confiance de $\frac{\text{support}(\{1,2,3\})}{\text{support}(\{1\})} = 33.33\%$, ce qui est inférieur au seuil de 50%, donc cette règle est rejetée.
   — La règle $\{1, 2\} \rightarrow \{3\}$ a une confiance de $\frac{\text{support}(\{1,2,3\})}{\text{support}(\{1,2\})} = 50\%$, ce qui est égal ou supérieur au seuil de 50%, donc cette règle est acceptée.

   On applique cette méthode à toutes les règles possibles pour identifier celles qui sont les plus significatives et utiles pour l'analyse des données transactionnelles.

II.3.2. L'algorithme ARMADA

II.3.2.1. Contexte et origine

L'algorithme ARMADA [44] (Association Rule Mining And Deduction Algorithm) est une méthode avancée pour la découverte des règles d'association temporelles. Cet algorithme a été développé pour surmonter certaines des limitations des méthodes traditionnelles de fouille de données, telles que l'algorithme Apriori, en fournissant une approche plus efficace et adaptée aux bases de données temporelles. ARMADA a été conçu pour traiter les données séquentielles et temporelles, permettant ainsi de découvrir des motifs complexes qui évoluent dans le temps.

II.3.2.2. Principe de fonctionnement

L'algorithme ARMADA repose sur la découverte des règles d'association temporelles en se basant sur des intervalles de temps. Les étapes principales de l'algorithme sont les suivantes :

1. **Prétraitement des données :**
   — Les données brutes sont d'abord transformées en une forme appropriée pour l'analyse temporelle. Cela implique la conversion des horodatages en intervalles de temps discrets et la segmentation des données en périodes d'intérêt.
2. **Génération des itemsets fréquents :**
   — ARMADA utilise une approche similaire à Apriori pour générer des itemsets fréquents, mais avec des adaptations pour prendre en compte les dimensions temporelles. Les itemsets sont formés en tenant compte des contraintes temporelles définies par l'utilisateur.
3. **Découverte des règles d'association temporelles :**
   — Une fois les itemsets fréquents identifiés, l'algorithme génère des règles d'association en considérant les intervalles de temps pendant lesquels ces itemsets apparaissent fréquemment. Les règles sont ensuite évaluées en termes de support, confiance et pertinence temporelle.

II.3.2.3. Pseudocode de l'algorithme ARMADA

L'algorithme ARMADA suit les étapes suivantes pour générer les règles d'association temporelles :

Entrée: D : Base de données temporelle, $minsup$: Seuil de support
Sortie : Tous les motifs temporels normalisés fréquents
begin
  Lire D dans MDB (base de données en mémoire) pour trouver tous les états fréquents;
  for *chaque état fréquent s* **do**
    former un motif $q = \{s\}$;
    sortir q;
    construire q-idx = CreateIndexSet(s, $\{\}$, MDB);
    appeler MineIndexSet(q, q-idx);
  endfor
end

Algorithm 2: Pseudocode de l'algorithme ARMADA

II.3.3. L'algorithme Discovering During-Temporal Patterns (DTP)

L'algorithme Discovering During-Temporal Patterns (DTP) a été développé par Li Zhang, Guoqing Chen, Tom Brijs et Xing Zhang en 2008. Cet algorithme se concentre sur la découverte des motifs temporels, particulièrement ceux impliquant une relation de type "durant" dans les grandes bases de données temporelles [45]. Ces relations temporelles sont courantes et potentiellement précieuses dans de nombreuses applications réelles, telles que la prévision météorologique, la gestion de projet et l'analyse des comportements des patients en médecine.

II.3.3.1. Principe de fonctionnement

L'algorithme DTP (During-Temporal Pattern) est présenté comme une méthode itérative incluant des phases de jonction et d'élagage pour identifier les motifs temporels fréquents. L'algorithme commence par trouver les motifs d'état unique fréquents , puis génère des motifs candidats de longueur k en utilisant les motifs fréquents de longueur k-1. Les propriétés de transitivité et de sous-motifs fréquents sont exploitées pour optimiser le processus de fouille des données.

Phases de l'Algorithme

- **Phase de Jonction** : Les motifs fréquents sont utilisés pour générer de nouveaux candidats en ajoutant des états à des motifs existants. Les propriétés 3 et 4 sont utilisées pour s'assurer que seuls les motifs valides sont ajoutés.
- **Phase d'Élagage** : Les motifs candidats dont le degré de support est inférieur au seuil spécifié sont éliminés. Les motifs restants sont conservés pour les itérations suivantes.

1. **Formulation du problème :**
   - Soit une base de données temporelle DT contenant N enregistrements sous la forme $\{a,(st,et)\}$, où a est l'état impliqué dans l'événement et (st,et) est l'intervalle de temps indiquant le début (st) et la fin (et) de l'état a.
2. **Définition des motifs temporels durant :**
   - Un événement $e_l = (a_i,t_l)$ est dit *durant* un autre événement $e_k = (a_j,t_k)$, noté $e_l \prec_d e_k$, si $S(e_l) \geq S(e_k)$ et $E(e_l) \leq E(e_k)$.
   - Un motif temporel durant (DTP) de longueur $k-1$ est de la forme $a_1 \prec_d a_2 \prec_d \ldots \prec_d a_k$.

II.3.3.2. Pseudocode de l'algorithme DTP

L'algorithme suit les étapes suivantes pour découvrir les motifs temporels durant dans une base de données temporelle :

Entrée: D_T : Base de données temporelle, α : Seuil de support, β : Seuil de confiance

Sortie : Q : Ensemble de motifs temporels avec confiance $\geq \beta$

```
begin
    PAO ← generate_PAO(D_T);
    frequent_patterns ← [];
    // Étape 1 : Génération des PAO fréquentes
    for pattern ∈ PAO do
        if calculate_support(pattern) ≥ α then
            frequent_patterns.append(pattern);
        end
    endfor
    // Étape 2 : Génération des motifs candidats
    candidate_patterns ← generate_candidates(frequent_patterns);
    // Étape 3 : Calcul du support et de la confiance
    for candidate ∈ candidate_patterns do
        support ← calculate_support(candidate);
        if support ≥ α then
            confidence ← calculate_confidence(candidate);
            if confidence ≥ β then
                frequent_patterns.append(candidate);
            end
        end
    endfor
    // Étape 4 : Élagage des motifs non fréquents
    prune_non_frequent(frequent_patterns);
    return frequent_patterns;
end
```

Algorithm 3: Pseudocode pour l'extraction de motifs temporels

L'algorithme DTP offre une approche puissante pour la découverte de motifs temporels complexes dans de grandes bases de données temporelles, fournissant des insights précieux pour une variété d'applications pratiques.

II.3.4. Algorithme proposé par Xiaoxiao et al. [1]

II.3.4.1. Principe de fonctionnement

L'algorithme proposé pour la découverte des motifs multi-temporels se base sur la transformation des données brutes en une forme adaptée à l'analyse temporelle. Le processus inclut la conversion des horodatages en intervalles de temps discrets et la segmentation des données en périodes d'intérêt. Ensuite, il suit les étapes suivantes pour générer les motifs multi-temporels :

1. **Prétraitement des données :**
   — Les données brutes sont transformées en une forme appropriée pour l'analyse temporelle. Cela implique la conversion des horodatages en intervalles de temps discrets et la segmentation des données en périodes d'intérêt.
2. **Génération des itemsets fréquents :**
   — L'algorithme utilise une approche similaire à Apriori pour générer des itemsets fréquents, mais avec des adaptations pour prendre en compte les dimensions temporelles. Les itemsets sont formés en tenant compte des contraintes temporelles définies par l'utilisateur.
3. **Découverte des modèles multi-temporels :**
   — Une fois les itemsets fréquents identifiés, l'algorithme génère des motifs en considérant les intervalles de temps pendant lesquels ces itemsets apparaissent fréquemment. Les motifs sont ensuite évalués en termes de support, de confiance et de pertinence temporelle.

II.3.4.2. Pseudocode de l'algorithme

Entrée: D_T : Base de données temporelle, α : Seuil de support, β : Seuil de confiance

Sortie : Q : Ensemble de modèles multi-temporels avec confiance $\geq \beta$

$S = \{s \mid s \text{ est un état dans } D_T\}$;
$C_0 = \{s_j \mid s_j \in S\}$;
$F_0 = \emptyset$;
$m = |S|$;
for $s_j \in S$ **do**
  $F_0 = F_0 \cup \{s_j \mid support(s_j) \geq \alpha, s_j \in S\}$;
endfor
$C_1 = \{s_i \xrightarrow{R} s_j \mid s_i \in S, s_j \in F_0\}$;
$k = 1$;
while $C_k \neq \emptyset$ **do**
  $F_k = \emptyset$;
  for $\phi_k \in C_k$ **do**
    Calcul du $support(\phi_k)$;
    if $support(\phi_k) \geq \alpha$ **then**
      $F_k = F_k \cup \{\phi_k\}$;
    end
  endfor
  for $\Phi_1 \in F_k$ **do**
    if Φ_1 *contient D ou E* **then**
      Flag1;
    end
    for $\Phi_2 \in F_k$ **do**
      if Φ_2 *correspond à* Φ_1 *ET* $\Phi_3 \in F_k$ **then**
        $C_{k+1} = C_{k+1} \cup \Phi'$;
      end
    endfor
    Filtrer C_{k+1};
  endfor
  $k = k + 1$;
end

Algorithm 4: Pseudocode pour l'extraction de modèles multi-temporels

Entrée: F_k : Ensemble de motifs fréquents, β : Seuil de confiance
Sortie : Q : Ensemble de modèles multi-temporels avec confiance $\geq \beta$
$Q = \emptyset$;
$k = 0$;
while $F_k \neq \emptyset$ **do**
  for $\phi \in F_k$ **do**
    Calcul de $confidence(\phi)$;
    if $confidence(\phi) \geq \beta$ **then**
      $Q = Q \cup \{\phi\}$;
    end
  endfor
  $k = k + 1$;
end

Algorithm 5: Pseudocode pour le calcul de la confiance des modèles multi-temporels

Étant donné une base de données temporelle D_T avec l'ensemble des états S, initialement, un ensemble de motifs dégénérés (k = 0) pourrait être construit, par exemple, $C_0 = \{s_j | s_j \in S\}$. Ensuite, la base de données temporelle pourrait être scannée pour déterminer si chaque s_j est fréquent ou non. Ainsi, un ensemble de motifs dégénérés 1-item (k = 0) pourrait être obtenu, par exemple, $F_0 = \{s_j | support(s_j) \geq \alpha, s_j \in S\}$. Supposons qu'il existe n événements dans la base de données temporelle, la complexité de calcul de ce processus est $O(n)$, c'est-à-dire, un scan de la base de données pour calculer les valeurs de support de tous les états.

En outre, basé sur la propriété suivante, les motifs fréquents à 2-items (k = 1) peuvent être générés basés sur les motifs à 1-item, ainsi de suite.

II.3.4.3. Exemple de relation temporelle

Pour illustrer l'application de l'algorithme, considérons une relation temporelle spécifique que cet article cherche à déduire : par exemple, *X augmente avant que Y diminue*. Cela signifie que l'état $\langle X, price, increase \rangle$ se produit avant l'état $\langle Y, price, decrease \rangle$.

II.3.4.4. Prédicats temporels

Les relations temporelles entre les événements peuvent être catégorisées en plusieurs types de prédicats de comparaison temporelle, tels que :

— **Before (B)** : Un événement e_A se produit avant un autre événement e_B si $et_A < st_B$.

— **During (D)** : Un événement e_A se produit pendant un autre événement e_B si $st_B \leq st_A$ et $et_A \leq et_B$.
— **Equal (E)** : Deux événements e_A et e_B sont égaux si $st_A = st_B$ et $et_A = et_B$.
— **Overlap (O)** : Deux événements e_A et e_B se chevauchent si $st_A < st_B < et_A$ ou $st_B < st_A < et_B$.

II.3.4.5. Faits temporels (augmentation et diminution)

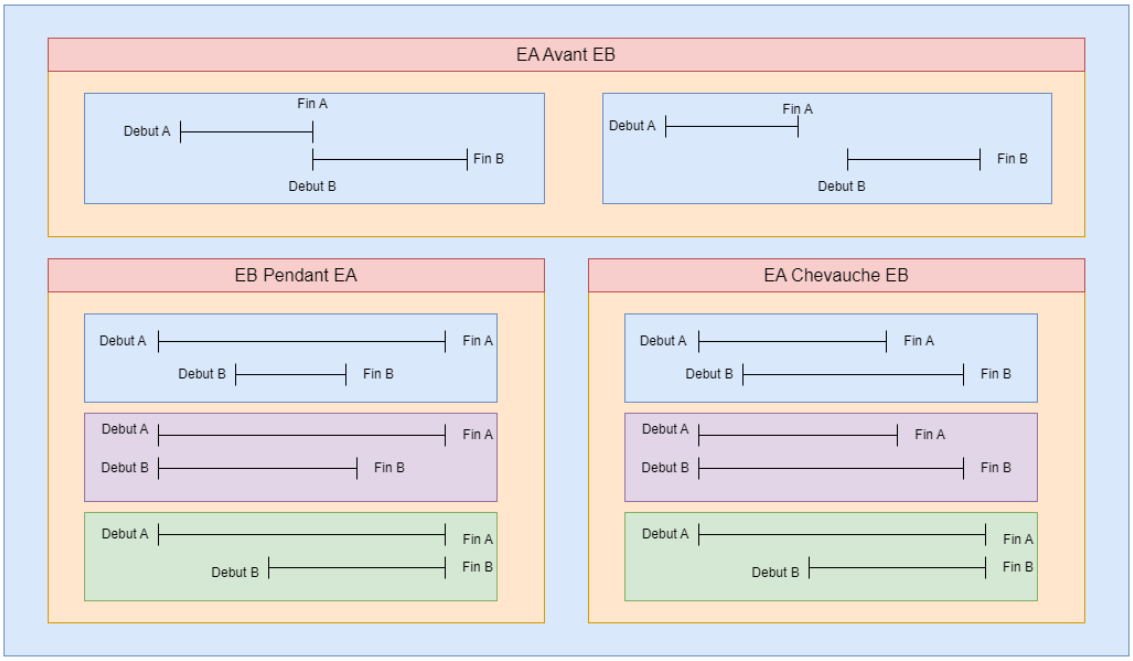
Les faits temporels étudiés dans cet article incluent des événements tels que l'augmentation et la diminution des prix des actions. Par exemple :

— **Augmentation (Increase)** : Un état $s = \langle X, price, increase \rangle$ signifie que le prix de l'action X augmente.
— **Diminution (Decrease)** : Un état $s = \langle Y, price, decrease \rangle$ signifie que le prix de l'action Y diminue.

II.4. Limites de recherche

Bien que l'algorithme présenté par Kong et al. dans l'article [1] soit capable d'extraire des modèles multi-temporels, il présente également des limitations significatives :

— **Complexité exponentielle et temps d'exécution :** L'un des principaux défis de l'algorithme proposé par Kong et al. est sa complexité exponentielle. En raison de la croissance rapide du nombre de combinaisons possibles à explorer dans les données, l'algorithme nécessite un temps d'exécution considérable pour produire des résultats. Cela limite son applicabilité pratique, surtout pour les ensembles de données de grande taille ou ceux nécessitant une analyse en temps réel.
— **Limitations des prédicats temporels :** Les prédicats temporels utilisés dans l'algorithme de Kong et al., notamment "avant", "pendant" et "chevauche", ne permettent pas toujours de capturer de manière précise et expressive la sémantique des modèles extraits. Tout ces prédicats peuvent manquer de finesse pour représenter des relations temporelles complexes entre les événements, réduisant ainsi la pertinence et la qualité des motifs découverts. Par conséquent, il y a une limitation à la fois sur le plan de l'expression sémantique des motifs extraits et de leur utilité pratique.



II.5. Conclusion

En conclusion, ce chapitre a fourni une vue d'ensemble des modèles multi-temporels et de l'état de l'art dans ce domaine. Nous avons exploré les concepts de base des modèles multi-temporels, notamment les types de relations temporelles, les représentations formelles et les méthodes d'extraction. De plus, nous avons examiné les progrès récents dans le domaine, en mettant en évidence les différentes approches et techniques utilisées pour modéliser et analyser les données temporelles. En intégrant ces connaissances, nous sommes mieux préparés à aborder la contribution dans l'extraction des modèles multi-temporels dans divers domaines d'application. Ce chapitre jette ainsi les bases nécessaires pour la suite de notre exploration des méthodes et des outils pour la découverte et l'exploitation de motifs temporels.

III

CHAPITRE

EXTENSION DES PRÉDICATS, UTILISATION DES MATRICES ET ALGORITHME PARALLÈLE POUR L'EXTRACTION DES MODÈLES MULTI-TEMPORELS

SOMMAIRE

III.1 - Introduction 55
III.2 - Amélioration des prédicats 56
III.3 - Méthode de prétraitement des données 58
III.4 - Amélioration de l'algorithmiques d'extraction des modèles multi-temporels 61
III.5 - Expérimentations et résultats 70
III.6 - Synthèse 74

III.1. Introduction

Les algorithmes étudiés dans la littérature et adaptés aux modèles multi-temporels sont ceux sur lesquels nous nous sommes appuyés pour développer notre approche dans cette partie. Parmi ces algorithmes, nous avons examiné des techniques telles que Apriori [22], ARMADA [44], DTP [45], et l'algorithme proposé par Xiaoxiao Kong et al. [1]. Dans cette section, nous allons présenter notre travail, qui se base principalement sur l'algorithme proposé par Xiaoxiao Kong et al., ainsi que les résultats obtenus et des perspectives d'amélioration.

Cette présentation se fera suivant trois principaux points. Le premier point porte sur les extensions de prédicats que nous avons ajoutées à l'algorithme de Xiaoxiao Kong pour améliorer la capture des modèles multi-temporels complexes. Ces extensions permettent de mieux représenter les relations temporelles entre les événements.

Le deuxième point aborde la vectorisation des opérations de calcul du support. Cette optimisation a pour objectif d'améliorer l'efficacité et la rapidité de l'algorithme, en réduisant le temps de traitement nécessaire pour analyser de grands volumes de données temporelles.

Enfin, le troisième point, et le plus important, présente une approche basée sur la parallélisation des opérations de calcul du support et de la confiance. En exploitant les architectures de calcul parallèle, nous visons à accélérer significativement le processus de découverte des modèles multi-temporels. Cette parallélisation permet de traiter simultanément plusieurs segments de données.

III.2. Amélioration des prédicats

Avant de définir les prédicats temporels, il est important de comprendre certains termes clés utilisés dans notre contexte :

— **st** (start time) : Le moment de début d'un événement.
— **et** (end time) : Le moment de fin d'un événement.
— **s** : Un état.
— **e** : Un événement, défini par un état et un intervalle de temps associé.

Définitions des prédicats temporels selon Xiaoxiao Kong et al.

Un prédicat temporel est une relation qui décrit comment deux événements se situent dans le temps l'un par rapport à l'autre. Les prédicats temporels permettent de déterminer les relations temporelles entre des événements en utilisant leurs intervalles de temps (début et fin).

Dans l'article de Xiaoxiao Kong et al. [1], les prédicats temporels utilisés pour composer une instance temporelle entre deux événements A et B sont définis comme suit :

1. Si $st_A = st_B$ et $et_A = et_B$, alors e_A égale e_B.
2. Si $0 \leq st_B - et_A \leq$ window, alors e_A avant e_B.
3. Si $st_B < st_A < et_A \leq et_B$ ou $st_B \leq st_A < et_A < et_B$, alors e_A pendant e_B.
4. Si $st_A \leq st_B < et_A < et_B$ ou $st_A < st_B < et_A \leq et_B$, alors e_A chevauche e_B.

Amélioration avec les relations temporelles d'Allen

Ces précédents prédicats peuvent manquer de granularité pour capturer toutes les relations temporelles complexes. Pour améliorer la capture des modèles multi-temporels, nous avons intégré les intervalles de temps d'Allen, qui offrent une description plus fine des relations entre les événements.

Les relations temporelles d'Allen [46] et leurs expressions mathématiques sont les suivantes :

— e_A avant e_B : $et_A < st_B$
— e_A rencontre e_B : $et_A = st_B$
— e_A chevauche e_B : $st_A < st_B < et_A < et_B$
— e_A est fini par e_B : $st_A < st_B \leq et_A = et_B$
— e_A contient e_B : $st_A < st_B$ et $et_B < et_A$
— e_A commence e_B : $st_A = st_B$ et $et_A < et_B$
— e_A égale e_B : $st_A = st_B$ et $et_A = et_B$

Ces nouveaux prédicats permettent de décrire plus précisément les relations entre les événements. Par exemple, au lieu de simplement dire qu'un événement chevauche un autre, nous pouvons spécifier s'il est "chevauché par" ou "fini par" l'autre événement, offrant ainsi une meilleure compréhension des modèles multi-temporels.

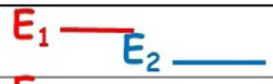
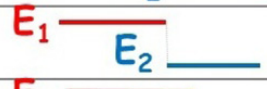
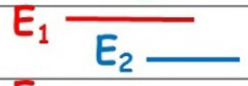
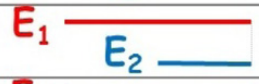
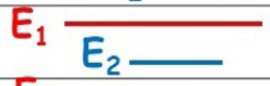
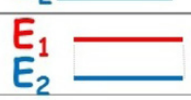
	E_1 before E_2	E_2 after E_1
	E_1 meets E_2	E_2 is-met-by E_1
	E_1 overlaps E_2	E_2 is-overlapped-by E_1
	E_1 is-finished-by E_2	E_2 finishes E_1
	E_1 contains E_2	E_2 during E_1
	E_1 starts E_2	E_2 is-started-by E_1
	E_1 equals E_2	E_2 equals E_1

FIGURE 18 – *Relations temporelles selon les intervalles de temps d'Allen [11]*

L'utilisation des relations temporelles d'Allen enrichit considérablement notre capacité à capturer et analyser les modèles multi-temporels complexes dans les grandes bases de données temporelles. Cette amélioration nous permet de mieux

comprendre les interactions temporelles et d'extraire des connaissances plus précises et significatives.

III.3. Méthode de prétraitement des données

Le prétraitement des données est une étape cruciale pour garantir l'efficacité et la précision des algorithmes de fouille de modèles multi-temporels. Inspiré de [1], cette section décrit notre approche pour transformer et préparer les données brutes en vue de l'analyse.

Objectifs du prétraitement

Le prétraitement des données a pour but de :

— Convertir les dates et les heures en indices numériques pour faciliter les calculs.

— Structurer les données de manière à permettre une extraction efficace des modèles multi-temporels.

Algorithme de prétraitement

L'algorithme de prétraitement comprend plusieurs étapes, que nous décrivons ci-dessous en utilisant un pseudocode algorithmique.

Étape 1 : Chargement des données

Nous commençons par charger les données brutes dans une structure appropriée, par exemple une table ou un tableau.

Date	Object	...	Valeur
14-07-2001	Element1	...	150
14-07-2001	Element2	...	17
...	...	...	...
15-07-2001	Element1	...	132

FIGURE 19 – *Chargement des données brutes*

Étape 2 : Regrouper les elements identiques par groupe

À cette étape, nous regroupons les événements similaires en fonction de leur nature et de leurs valeurs, ce qui permet de former des événements structurés et exploitables pour l'analyse temporelle

Evènement A (Exple : Bitcoin)			
N°	Jour	Valeur	Evènements merges
1	10-07-2023	120	{Evènement A augmente, 1, 3}
2	11-07-2023	123	
3	12-07-2023	127	
4	13-07-2023	115	{Evènement A diminue, 3, 5}
5	14-07-2023	114	
6	15-07-2023	135	{Evènement A augmente, 5, 8}
7	16-07-2023	136	
8	17-07-2023	137	
9	18-07-2023	136	{Evènement A diminue, 8, 12}
10	19-07-2023	132	
11	20-07-2023	131	
12	21-07-2023	124	
...	...	...	...

FIGURE 20 – *Conversion des données en événements*

Evènement B (Exple : Etheruim)			
N°	Jour	Valeur	Evènements merges
1	10-07-2023	54	{Evènement B augmente, 1, 3}
2	11-07-2023	55	
3	12-07-2023	57	{Evènement B diminue, 3, 4}
4	13-07-2023	55	
5	14-07-2023	57	{Evènement B augmente, 4, 6}
6	15-07-2023	54	
7	16-07-2023	53	
8	17-07-2023	52	
9	18-07-2023	51	{Evènement B diminue, 6, 12}
10	19-07-2023	50	
11	20-07-2023	45	
12	21-07-2023	40	
...	...	...	...

FIGURE 21 – *Conversion des données en événements*

Étape 3 : Affichage et sauvegarde des données

Nous affichons et sauvegardons les données prétraitées pour une analyse ultérieure.

e_A **augmente**	e_A **diminue**	e_B **augmente**	e_B **diminue**	**...**	**...**
(1, 3)	(3, 8)	(1, 3)	(3, 4)	...	...
(5, 8)	(8, 12)	(4, 6)	(6, 12)	...	...
...	...	...	...	...	...

TABLE VII – *Résultat tableau temporelle nettoyé*

Le code source est le suivant

```python
import pandas as pd


data = pd.read_csv('data.csv')
# Conversion du type de la colonne 'date'
data['date'] = pd.to_datetime(data['date'])


# Identification des états "augmentation" et "diminution"
data['state'] = data['name'] + "-" + data['close'].diff().apply(lambda x: "increase" if x > 0 else "decrease")

# Création de la base de données temporelle
events = []
for name, group in data.groupby('name'):
    start_date = None
    current_state = None
    for index, row in group.iterrows():
        if current_state is None or row['state'] != current_state:
            if current_state is not None:
                events.append((current_state, start_date, row['date']))
            start_date = row['date']
            current_state = row['state']
    if current_state is not None:
        events.append((current_state, start_date, row['date']))

temporal_db = pd.DataFrame(events, columns=['State', 'st', 'et'])

# Affichage de la base de données temporelle sous forme de tableau
temporal_db['st'] = temporal_db['st'].dt.strftime('%Y-%m-%d')
temporal_db['et'] = temporal_db['et'].dt.strftime('%Y-%m-%d')
print(temporal_db)
```

FIGURE 22 – *Debut code source*

```python
import pandas as pd

# Sample data
data = temporal_db

# Function to convert dates to numeric indices
def date_to_index(data):
    date_indices = {date: idx+1 for idx, date in enumerate(sorted(data['st'].unique()))}
    data['st_idx'] = data['st'].map(date_indices)
    data['et_idx'] = data['et'].map(date_indices)
    return data

# Convert dates to indices
data = date_to_index(data)

# Create the interval column
data['Interval'] = data.apply(lambda row: [row['st_idx'], row['et_idx']], axis=1)

# Function to merge overlapping intervals
def merge_intervals(intervals):
    sorted_intervals = sorted(intervals, key=lambda x: x[0])
    merged = []
    for current in sorted_intervals:
        if not merged or merged[-1][1] < current[0]:
            merged.append(current)
        else:
            merged[-1][1] = max(merged[-1][1], current[1])
    return [f"({start}, {end})" for start, end in merged if start != end]

# Group by State and merge intervals
grouped_data = data.groupby('State')['Interval'].apply(list).to_dict()
merged_intervals = {state: merge_intervals(intervals) for state, intervals in grouped_data.items()}

# Determine the maximum number of intervals for any state
max_intervals = max(len(intervals) for intervals in merged_intervals.values())

# Create a dictionary for the formatted data
formatted_data = {state: intervals + [''] * (max_intervals - len(intervals)) for state, intervals in merged_intervals.items()}

# Create the final DataFrame
result_df = pd.DataFrame(formatted_data)

# Display the final table
print(result_df)
result_df.to_excel('outputDateNumber.xlsx', index=False)
```

FIGURE 23 – *Fin code source*

Ce processus de prétraitement garantit que les données sont correctement préparées pour l'analyse des modèles multi-temporels, permettant ainsi d'améliorer la précision et l'efficacité des algorithmes de fouille de données utilisés dans notre étude.

III.4. Amélioration de l'algorithmiques d'extraction des modèles multi-temporels

Dans l'algorithme proposé par Xiaoxiao Kong et al. [1], la complexité d'exécution de cet algorithme est de l'ordre de $O(n \cdot m^{k+1})$, où n est le nombre total d'événements, m est le nombre d'états distincts, et k est le nombre de prédicats temporels dans le motif. Cette complexité est due à la fois à la formation des modèles et au calcul du support et de la confiance des différents motifs générés.

Nous nous concentrerons ici sur l'optimisation du calcul du support et de la confiance des modèles.

Exemple de calcul du support pour le prédicat avant

Considérons le calcul du support du motif "Cardano-diminue avant Bitcoin-augmente" avec les intervalles de temps suivants :

— Intervalles de diminution de Cardano : (1444, 1445), (1446, 1447)

— Intervalles d'augmentation de Bitcoin : (2, 3), (4, 5), (6, 7), (8888, 9888), (1990, 1991), (1299, 1885)

Le tableau ci-dessous montre les comparaisons nécessaires pour déterminer le support :

Cardano-diminue < Bitcoin-augmente	**Comparaison**
1445 < 2	Faux
1445 < 4	Faux
1445 < 6	Faux
1445 < 8888	Vrai
1445 < 1990	Vrai
1445 < 1299	Faux
1447 < 2	Faux
1447 < 4	Faux
1447 < 6	Faux
1447 < 8888	Vrai
1447 < 1990	Vrai
1447 < 1299	Faux

TABLE VIII – *Comparaison des intervalles de temps*

Nombre de paires satisfaisant "avant" : 4
Nombre total de paires possibles : $2 \times 6 = 12$

Support = $\frac{4}{12} = \frac{1}{3} \approx 0.333$

Optimisation par vectorisation et parallélisation

En analysant ce processus, nous constatons que pour calculer le support d'un modèle avec m éléments d'augmentation et n éléments de diminution, on effectue séquentiellement $m \times n$ comparaisons, ce qui est très coûteux en termes de temps.

Pour pallier cette inefficacité, nous proposons une combinaison de deux approches :

1. **Vectorisation** : Utiliser des matrices pour représenter les intervalles de temps et calculer le support de manière plus efficace. Cela permet de paralléliser les opérations de comparaison.
2. **Parallélisation** : Exploiter les architectures de calcul parallèle pour distribuer les opérations de calcul du support et génération de models temporels sur les différents cœurs du processeur.

III.4.1. Utilisation de matrices

Le calcul du support peut être formulé comme une opération matricielle pour optimiser le processus. En représentant les intervalles de temps comme des matrices, nous pouvons utiliser des opérations vectorielles pour effectuer des comparaisons de manière plus efficace.

Formulation du calcul du support comme un problème matriciel

Pour optimiser le calcul du support, les intervalles de temps des événements sont convertis en matrices. Supposons que nous avons deux ensembles d'événements : A et B. Les intervalles de temps pour ces événements peuvent être représentés par les matrices suivantes :

Soit Matrix_A représentant les intervalles de temps des événements A et Matrix_B représentant les intervalles de temps des événements B :

$$\text{Matrix}_A = \begin{pmatrix} t_{A1}^{start} & t_{A1}^{end} \\ t_{A2}^{start} & t_{A2}^{end} \\ \vdots & \vdots \\ t_{An}^{start} & t_{An}^{end} \end{pmatrix}, \quad \text{Matrix}_B = \begin{pmatrix} t_{B1}^{start} & t_{B1}^{end} \\ t_{B2}^{start} & t_{B2}^{end} \\ \vdots & \vdots \\ t_{Bm}^{start} & t_{Bm}^{end} \end{pmatrix}$$

Pour comparer ces intervalles, nous construisons une matrice de comparaison Comparison Matrix qui vérifie la relation temporelle spécifiée **(avant, rencontre, chevauche, est fini par, contient, commence, égale)** entre chaque événement de la matrice A et chaque événement de la matrice B :

1. Prédicat "avant"

Pour le prédicat "avant" (e_A avant e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{end} < t_{Bj}^{start} \\ 0 & \text{sinon} \end{cases}$$

2. Prédicat "rencontre"

Pour le prédicat "rencontre" (e_A rencontre e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{end} = t_{Bj}^{start} \\ 0 & \text{sinon} \end{cases}$$

3. Prédicat "chevauche"

Pour le prédicat "chevauche" (e_A chevauche e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{start} < t_{Bj}^{start} < t_{Ai}^{end} < t_{Bj}^{end} \\ 0 & \text{sinon} \end{cases}$$

4. Prédicat "est fini par"

Pour le prédicat "est fini par" (e_A est fini par e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{start} < t_{Bj}^{start} \leq t_{Ai}^{end} = t_{Bj}^{end} \\ 0 & \text{sinon} \end{cases}$$

5. Prédicat "contient"

Pour le prédicat "contient" (e_A contient e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{start} < t_{Bj}^{start} \text{ et } t_{Bj}^{end} < t_{Ai}^{end} \\ 0 & \text{sinon} \end{cases}$$

6. Prédicat "commence"

Pour le prédicat "commence" (e_A commence e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{start} = t_{Bj}^{start} \text{ et } t_{Ai}^{end} < t_{Bj}^{end} \\ 0 & \text{sinon} \end{cases}$$

7. Prédicat "égale"

Pour le prédicat "égale" (e_A égale e_B), la condition est :

$$\text{Comparison Matrix}_{ij} = \begin{cases} 1 & \text{si } t_{Ai}^{start} = t_{Bj}^{start} \text{ et } t_{Ai}^{end} = t_{Bj}^{end} \\ 0 & \text{sinon} \end{cases}$$

La matrice de support est alors remplie avec des 1 pour les comparaisons vraies et des 0 pour les comparaisons fausses. Le support est ensuite calculé en sommant les éléments de la matrice de support et en divisant par le nombre total de comparaisons possibles.

Formule du calcul du support

La formule du calcul du support en utilisant la représentation matricielle est donnée par :

$$\text{support}(\Phi) = \frac{\sum_{i=1}^{m} \sum_{j=1}^{n} \text{Comparison Matrix}_{ij}}{m \times n}$$

où m est le nombre d'intervalles de B et n est le nombre d'intervalles de A.

Exemple d'application

Pour illustrer l'utilisation de matrices, reprenons l'exemple de calcul du support :

Considérons les matrices suivantes représentant les événements "Bitcoin-augmente" et "Cardano-diminue" :

$$\text{Matrix}_A = \begin{pmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \\ 8888 & 9888 \\ 1990 & 1991 \\ 1299 & 1885 \end{pmatrix}, \quad \text{Matrix}_B = \begin{pmatrix} 1444 & 1445 \\ 1446 & 1447 \end{pmatrix}$$

La matrice de comparaison construite pour vérifier les relations "avant" est la suivante :

$$\text{Comparison Matrix} = \begin{bmatrix} 1445 < 2 & 1445 < 4 & 1445 < 6 & 1445 < 8888 & 1445 < 1990 & 1445 < 1299 \\ 1447 < 2 & 1447 < 4 & 1447 < 6 & 1447 < 8888 & 1447 < 1990 & 1447 < 1299 \end{bmatrix}$$

Cela donne la matrice de support remplie :

$$\text{Support Matrix} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Le calcul du support est alors donné par :

$$\text{support}(\Phi) = \frac{\sum_{i=1}^{2} \sum_{j=1}^{6} \text{Support Matrix}[i,j]}{12} = \frac{4}{12} = 0.333$$

Avantages de l'utilisation des matrices

Optimisation des Calculs Contrairement aux boucles imbriquées des méthodes traditionnelles, les opérations sur matrices permettent de comparer directement des intervalles de temps en une seule étape vectorisée, exploitant le principe de SIMD pour effectuer des calculs en parallèle. Cela réduit considérablement le temps de traitement.

Réduction des Passages sur les Données Les méthodes traditionnelles peuvent nécessiter plusieurs passages sur les données pour vérifier les conditions. L'approche matricielle effectue toutes les vérifications nécessaires en une seule passe, optimisant ainsi l'utilisation des ressources.

Simplicité et Clarté L'approche matricielle permet de représenter les opérations complexes de manière plus concise et lisible, facilitant ainsi la compréhension et la maintenance du code.

Capacités SIMD et Calcul du Nombre d'Opérations en Parallèle

Les unités SIMD présentes dans les processeurs modernes permettent d'effectuer plusieurs opérations en parallèle, augmentant ainsi l'efficacité des calculs. La capacité des unités SIMD est définie par la largeur du registre SIMD et le type de données traitées.

- **Largeur du Registre SIMD** :
  - SSE (Streaming SIMD Extensions) utilise des registres de 128 bits.
  - AVX (Advanced Vector Extensions) utilise des registres de 256 bits.
  - AVX-512 utilise des registres de 512 bits.
- **Taille des Données** : Le nombre d'opérations en parallèle dépend également de la taille des données que vous traitez. Par exemple, si vous traitez des données de 32 bits (4 octets), un registre de 128 bits peut traiter 4 éléments en parallèle.
- **Calcul du Nombre d'Opérations en Parallèle** : Pour déterminer le nombre d'opérations en parallèle, utilisez la formule suivante :

$$w = \frac{\text{Largeur du registre SIMD (en bits)}}{\text{Taille d'un élément (en bits)}}$$

Exemple Supposons que vous avez un processeur avec des registres AVX de 256 bits et que vous traitez des données de 32 bits (4 octets) :

- Largeur du registre SIMD : 256 bits

— Taille des données : 32 bits

Le nombre d'opérations en parallèle w est :

$$w = \frac{256}{32} = 8$$

Cela signifie que le SIMD peut traiter 8 opérations en parallèle.

Nous obtenons donc une complexité de $\mathbf{O}\left(\frac{n \cdot m^{k+1}}{w}\right)$, où n est le nombre total d'événements, m est le nombre d'états distincts, k est le nombre de prédicats temporels dans le motif, et w est le nombre d'opérations en parallèle dans le SIMD.

III.4.2. Parallélisation de l'algorithme

Pour améliorer encore l'efficacité de l'algorithme, nous parallélisons le bloc entier qui inclut le calcul du support et la génération des modèles multi-temporels. Nous utilisons les modules `joblib` et `concurrent.futures` de Python pour réaliser cette parallélisation en **environnement partagé**.

Mise en Œuvre de la Parallélisation

1. **Division des Modèles :** Les modèles sont répartis entre les cœurs du processeur. Chaque cœur traite environ $\frac{k}{p}$ modèles.
2. **Assignation des Tâches aux Threads :** Chaque cœur exécute le traitement des modèles, incluant calcul du support, comparaison des intervalles de temps, et vérification des conditions.
3. **Agrégation des Résultats :** Les résultats des cœurs sont agrégés pour obtenir les motifs valides et compiler les résultats finaux.

Définition des Ensembles et Notations

— S : L'ensemble initial des états extraits de la base de données temporelle D_T.
— C_0 : L'ensemble des modèles candidats de longueur 1, généré à partir des états de S.
— F_0 : L'ensemble des modèles fréquents de longueur 1, filtré à partir de C_0 en utilisant le seuil de support α.
— C_k : L'ensemble des modèles candidats de longueur k, généré à partir des modèles fréquents F_{k-1}.
— F_k : L'ensemble des modèles fréquents de longueur k, filtré à partir de C_k en utilisant le seuil de support α.

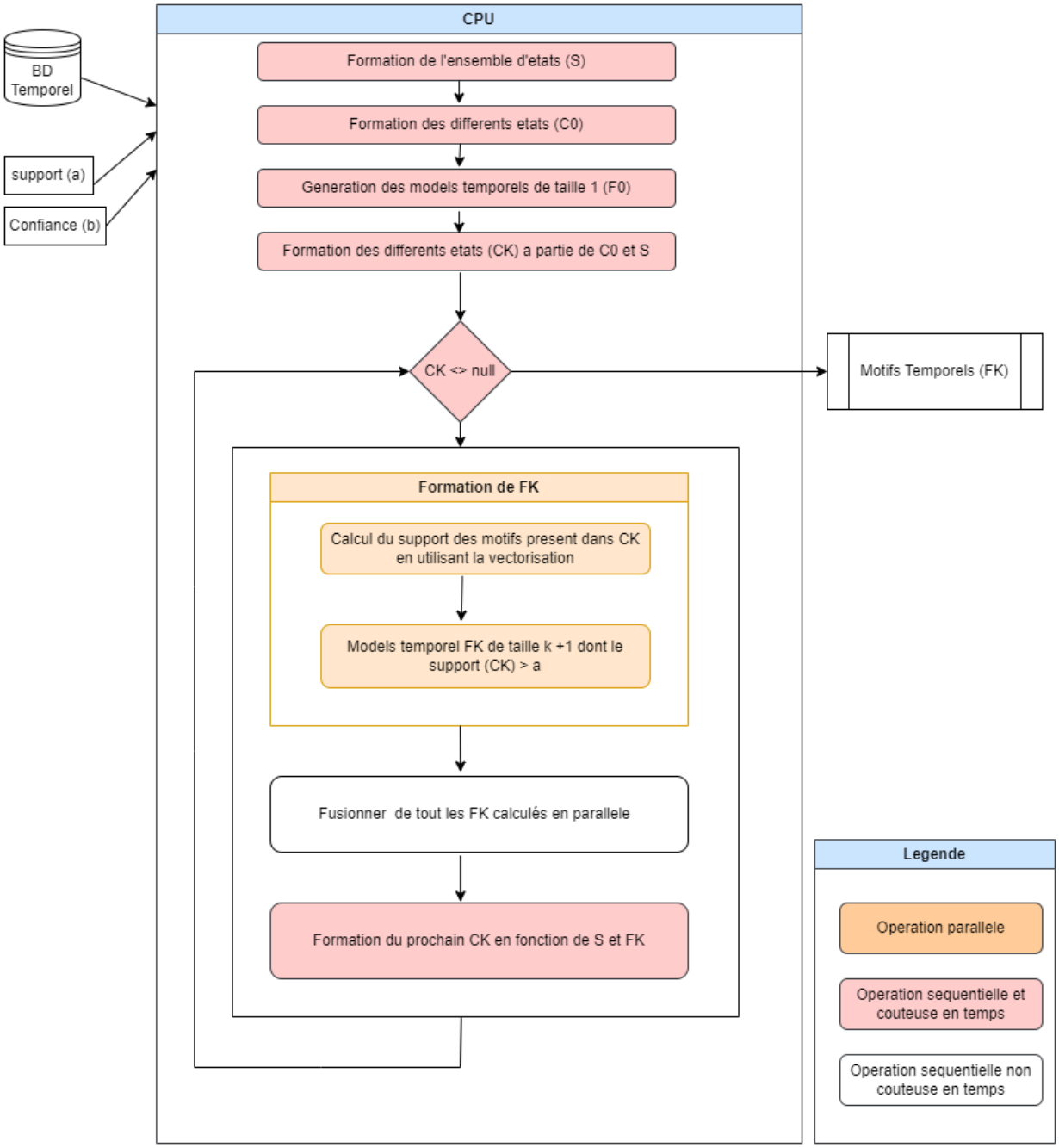


FIGURE 24 – *Étapes de l'algorithme*

Nous obtenons donc une complexité de $\mathbf{O}\left(\frac{n \cdot m^{k+1}}{pw}\right)$, où n est le nombre total d'événements, m est le nombre d'états distincts, k est le nombre de prédicats temporels dans le motif, p est le nombre de processeurs, et w est le nombre d'opérations en parallèle dans le SIMD.

Entrée: D_T : Base de données temporelle, α : Seuil de support, β : Seuil de confiance
Sortie : Q : Ensemble de modèles multi-temporels avec confiance $\geq \beta$
$S = \{s \mid s \text{ est un état dans } D_T\}$;
$C_0 = \{s_j \mid s_j \in S\}$;
$F_0 = \emptyset$;
$m = |S|$;
for $s_j \in S$ **do**
  $F_0 = F_0 \cup \{s_j \mid support(s_j) \geq \alpha, s_j \in S\}$;
endfor
$C_1 = \{s_i \xrightarrow{R} s_j \mid s_i \in S, s_j \in F_0\}$;
$k = 1$;
while $C_k \neq \emptyset$ **do**
  // Parallélisation du bloc entier
  $F_k = \emptyset$;
  for $\phi_k \in C_k$ **do in parallel**
    Calcul du $support(\phi_k)$;
    if $support(\phi_k) \geq \alpha$ **then**
      $F_k = F_k \cup \{\phi_k\}$;
    end
  end
  // Fusionner les résultats obtenus par chaque cœur
  $F_k = \bigcup_{i=1}^{p} F_k^{(i)}$ // Où $F_k^{(i)}$ est le résultat du i^{me} cœur
  for $\Phi_1 \in F_k$ **do**
    if Φ_1 *contient D ou E* **then**
      Flag1;
    end
    for $\Phi_2 \in F_k$ **do**
      if Φ_2 *correspond à* Φ_1 *ET* $\Phi_3 \in F_k$ **then**
        $C_{k+1} = C_{k+1} \cup \Phi'$;
      end
    endfor
    Filtrer C_{k+1};
  endfor
  $k = k + 1$;
end

Algorithm 6: Pseudocode pour l'extraction de modèles multi-temporels avec parallélisation

III.5. Expérimentations et résultats

III.5.1. Analyse des prédicats temporels

Pour tester notre algorithme, nous avons utilisé la base de données financière Wind, Shanghai Wind Information Co., Ltd. [47]. Toutes les données brutes, y compris les cours boursiers quotidiens, l'analyse sectorielle et les données d'indices, couvrent la période de juillet 2003 à janvier 2008, à l'exception de la période de janvier à juin 2003 pendant laquelle le SRAS (le Syndrome Respiratoire Aigu Sévère) était fortement épidémique. L'analyse se concentre sur les données des principaux indices sectoriels des marchés chinois et hongkongais ainsi que sur les principaux indices composites de ces marchés. Les résultats obtenus sont présentés ci-dessous.

Après plusieurs séries de tests, basés sur les données temporelles prétraitées, nous avons fixé $\alpha = 0.3$ et $\beta = 0.4$. Sur la base de données temporelles au niveau du marché, Les modèles multi-temporels découverts à différents niveaux sont présentés dans les tableaux ci-dessous. Les models incluent des prédicats "avant" (B) et "contient" (D).

TABLE IX – *Models temporels au niveau de l'industrie*

#	**Modèles multi-temporels au niveau de l'industrie**
1	Hang Seng Telecommunications Index augmente $\xrightarrow{B}$ CSI 300 Telecommunications Index augmente
2	Hang Seng Energy Index augmente $\xrightarrow{B}$ CSI 300 Energy Index augmente
3	Hang Seng Financials Index augmente $\xrightarrow{B}$ CSI 300 Energy Index augmente
4	Hang Seng Industrials Index augmente $\xrightarrow{B}$ CSI 300 Consumer Index augmente
5	Hang Seng IT Index augmente $\xrightarrow{B}$ CSI 300 Telecommunication Index augmente
6	Hang Seng IT Index augmente $\xrightarrow{B}$ CSI 300 IT Index augmente
7	Hang Seng Utilities Index augmente $\xrightarrow{B}$ CSI 300 Utilities Index augmente
8	CSI 300 Materials Index augmente $\xrightarrow{B}$ Hang Seng Materials Index augmente
9	CSI 300 Energy Index augmente $\xrightarrow{B}$ CSI 300 Materials Index augmente $\xrightarrow{B}$ Hang Seng Materials Index augmente
10	CSI 300 Consumer Index augmente $\xrightarrow{B}$ CSI 300 Materials Index augmente $\xrightarrow{B}$ Hang Seng Industrials Index augmente
11	CSI 300 Energy Index augmente $\xrightarrow{B}$ CSI 300 Materials Index augmente $\xrightarrow{B}$ Hang Seng Industrials Index augmente
12	CSI 300 Consumer Index augmente $\xrightarrow{B}$ CSI 300 Materials Index augmente
13	Hang Seng Consumer Index augmente $\xrightarrow{B}$ CSI 300 Materials Index augmente
14	Hang Seng Financials Index augmente $\xrightarrow{B}$ Hang Seng Telecommunications Index augmente $\xrightarrow{B}$ CSI 300 Industrials Index augmente
15	Hang Seng Financials Index augmente $\xrightarrow{B}$ Hang Seng Telecommunications Index augmente $\xrightarrow{B}$ CSI 300 Materials Index augmente

TABLE X – *Models temporels au niveau du marché*

#	**Modèles multi-temporels au niveau du marché**
1	HSCFI diminue $\xrightarrow{D}$ HSCACI diminue $\xrightarrow{B}$ SCI diminue
2	HSCFI augmente $\xrightarrow{D}$ HSCACI augmente $\xrightarrow{D}$ HSCEI augmente

III.5.2. Optimisation de l'algorithme

Les expérimentations ont été menées sur un nœud avec un processeur Intel(R) Xeon(R) avec 64 cœurs, 250 Go de mémoire RAM, et un disque racine de 175 Go. Nous avons utilisé le dataset de Wind Financial Databases [47], couvrant la période de juillet 2003 à janvier 2008.

Optimisation Matricielle

Pour améliorer l'efficacité de notre algorithme, nous avons utilisé une approche basée sur les matrices. Cette méthode permet de représenter les intervalles de temps comme des matrices et de calculer le support des motifs de manière plus efficace. En utilisant les opérations matricielles, nous avons pu réduire considérablement le temps de calcul nécessaire pour analyser les relations temporelles entre les différents états.

Le graphique ci-dessous illustre le temps d'exécution en fonction du nombre d'états pour l'algorithme optimisé par les matrices :

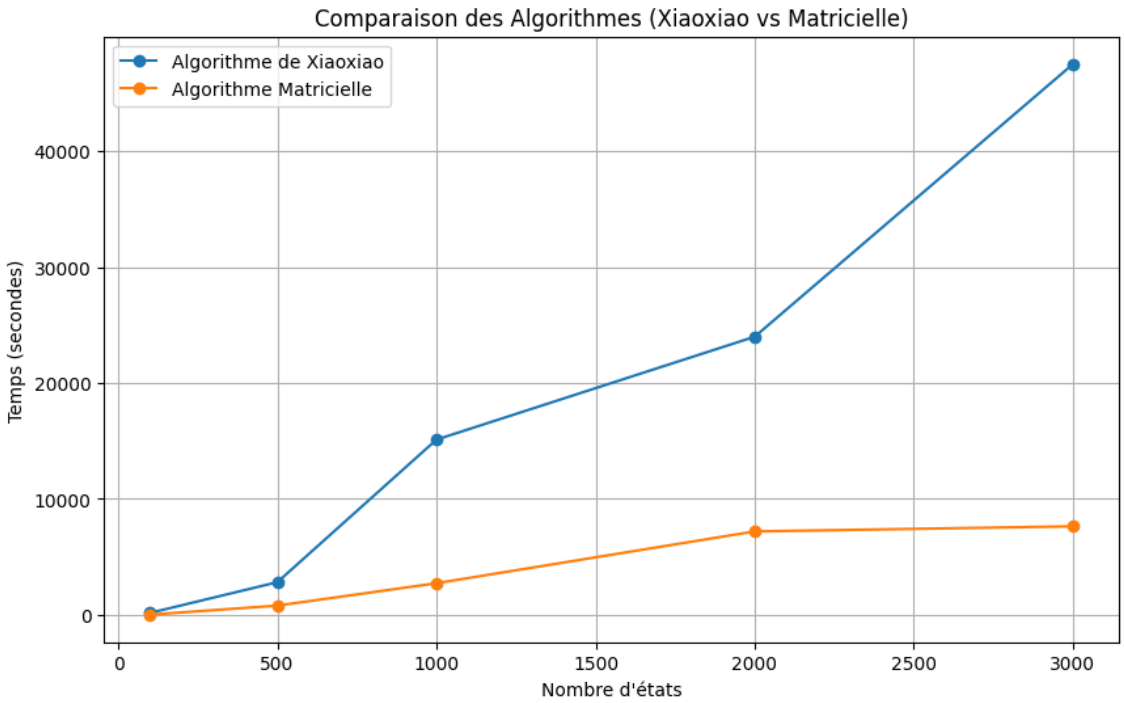


FIGURE 25 – *Temps d'exécution en fonction du nombre d'états.*

Le graphique montre une comparaison claire entre les deux algorithmes. L'algorithme de Xiaoxiao présente une croissance rapide du temps d'exécution avec l'augmentation du nombre d'états, indiquant une complexité exponentielle. En revanche, notre algorithme matriciel affiche une croissance plus modérée et linéaire du temps d'exécution. Cela démontre non seulement l'efficacité de notre méthode,

mais aussi sa capacité à gérer de grands volumes de données de manière plus stable et rapide. Cette optimisation est particulièrement bénéfique dans des scénarios où le traitement rapide de grandes bases de données temporelles est crucial. Nous constatons que notre approche matricielle peut être jusqu'à 8,4 fois plus rapide que l'algorithme de Xiaoxiao en fonction du nombre d'états.

Optimisation Parallèle

Pour améliorer encore l'efficacité de notre algorithme, nous avons intégré les bibliothèques `joblib` et `concurrent.futures` pour la parallélisation des tâches en environnement partagé. Cela permet d'exploiter pleinement les ressources CPU disponibles, réduisant ainsi le temps d'exécution global.

Évaluation du temps d'exécution

Pour évaluer le temps d'exécution de notre algorithme, nous avons comparé les performances avec et sans parallélisation sur différentes tailles de données. Les figures ci-dessous montrent les temps d'exécution pour différents nombres d'états et de threads.

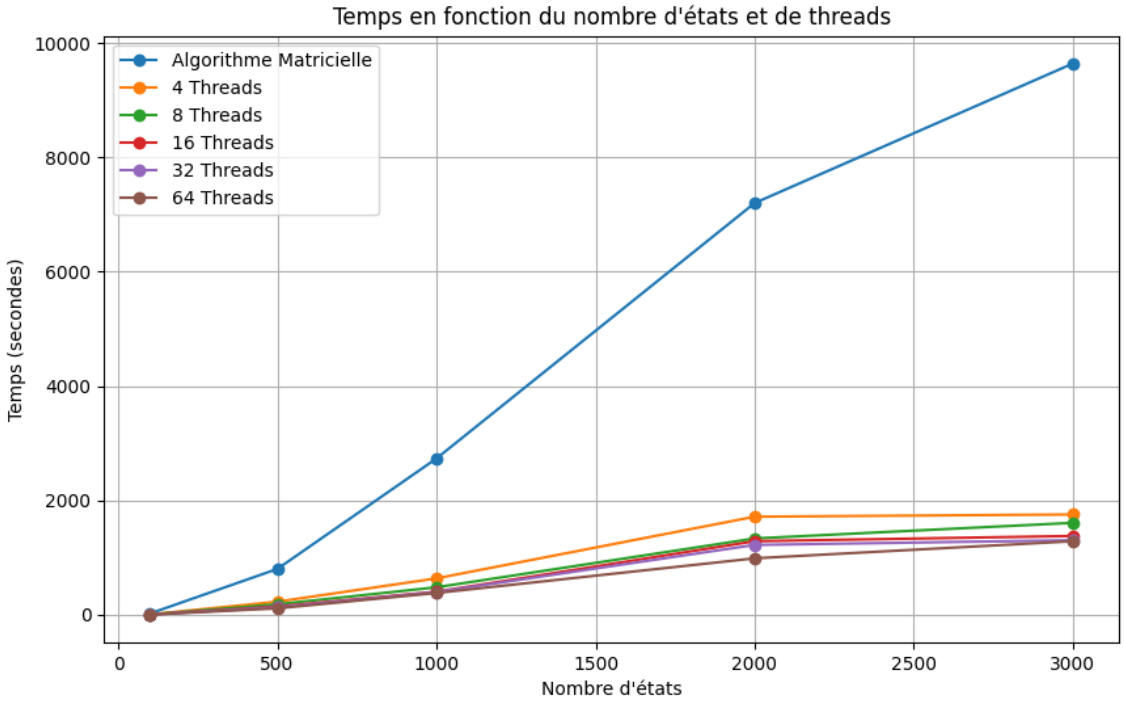


FIGURE 26 – *Temps d'exécution en fonction du nombre d'états sur différents threads.*

Les résultats montrent une amélioration significative des temps d'exécution grâce à la parallélisation. Le tableau ci-dessous résume les facteurs d'accélération obtenus pour différents nombres d'états en fonction du nombre de threads.

Algorithme Matricielle	Nombre de threads				
	4	8	16	32	64
Nombres d'états					
100	3.2	4.8	6.2	7.1	7.5
500	3.52	4.4	5.6	6.3	7.2
1000	3.6	5.7	6.8	6.9	7.2
2000	3.7	5.4	5.6	5.9	7.3
3000	3.7	6	7	7.4	7.5

TABLE XI – *Algorithme Matricielle - Accélération en fonction du nombre de threads*

Nous constatons que notre approche par parallélisation peut être jusqu'à 7,5 fois plus rapide que l'approche précédente. Nous avons également effectué une comparaison entre l'algorithme initial de Xiaoxiao et l'algorithme final obtenu.

Le graphique ci-dessous illustre le temps d'exécution en fonction du nombre d'états pour l'algorithme de Xiaoxiao et notre approche :

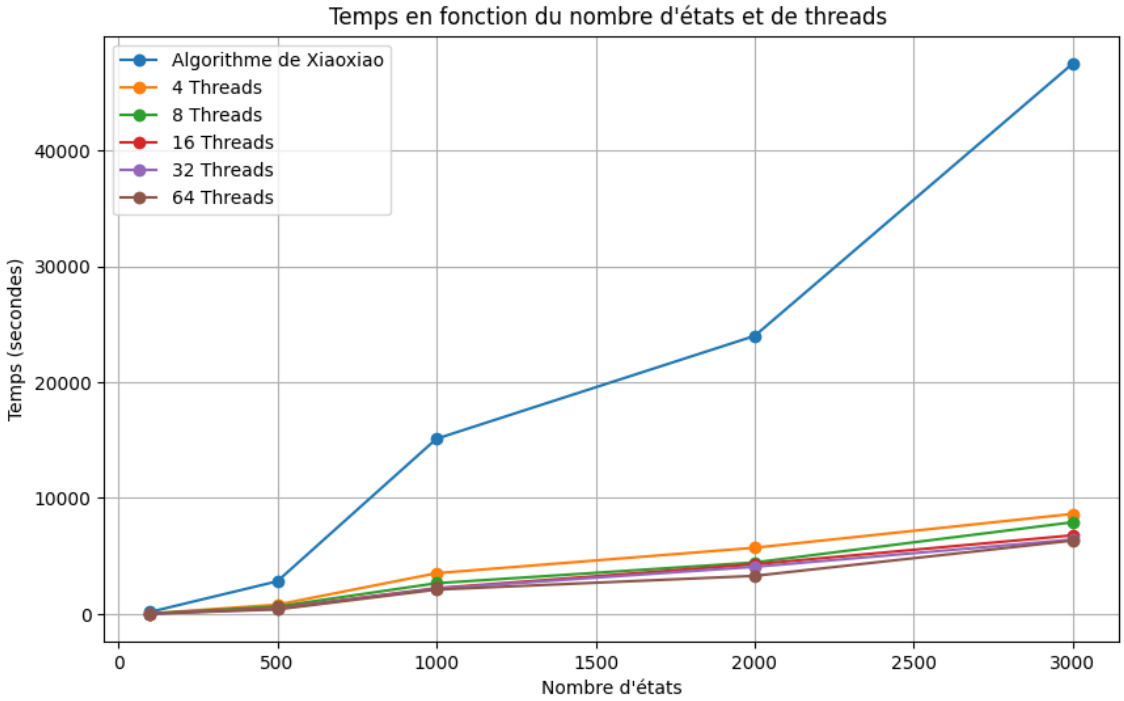


FIGURE 27 – *Comparaison des Algorithmes (Xiaoxiao vs Final)*

Les résultats montrent que notre algorithme est 15,9 fois plus rapide que celui de Xiaoxiao.

III.6. Synthèse

En conclusion, les résultats démontrent que l'optimisation par parallélisation et l'utilisation de matrices permettent de réduire significativement le temps d'exécution des algorithmes d'extraction des modèles multi-temporels. La parallélisation a permis d'augmenter l'efficacité de l'algorithme, surtout pour des ensembles de données de grande taille, tout en maintenant une bonne précision dans la découverte des modèles multi-temporels. Ces améliorations rendent notre approche plus adaptée aux applications pratiques nécessitant une analyse rapide et efficace des données temporelles.

CONCLUSION GÉNÉRALE

CONTENTS

Problématique étudiée et les choix méthodologiques 75
Analyse critique des résultats obtenus 76
Quelques perspectives 76

Problématique étudiée et les choix méthodologiques

L'évolution de l'informatique et des technologies de stockage a conduit à une explosion des volumes de données disponibles, particulièrement pertinentes pour des domaines tels que la finance, la santé et la surveillance. Ces données, souvent séquentielles et temporelles, nécessitent des méthodes avancées pour en extraire des motifs significatifs. L'analyse des données temporelles s'avère cruciale pour identifier des tendances, prédire des événements futurs et comprendre les relations complexes entre différentes variables au fil du temps.

Un modèle temporel permet de capturer les relations entre des séquences d'événements ordonnées dans le temps. Par exemple, un modèle peut représenter des événements tels que "le prix de l'action A a augmenté" avant que "le prix de l'action B n'augmente". Ces modèles sont largement utilisés dans des domaines où l'analyse temporelle est essentielle, comme les marchés financiers, la surveillance de la santé ou la prévision météorologique.

Dans ce contexte, notre recherche se concentre sur l'amélioration des algorithmes d'extraction des modèles temporels. Inspirés par les travaux de Xiaoxiao Kong et al. [1] et d'autres chercheurs, nous avons entrepris d'optimiser les prédicats temporels utilisés pour capturer des motifs complexes. Les relations temporelles traditionnelles, bien que efficaces, manquent parfois de granularité pour représenter fidèlement les interactions entre événements.

Nous avons donc étendu ces prédicats en intégrant les intervalles de temps d'Allen [46], offrant une description plus fine des relations entre événements. De plus, pour pallier au problème de lenteur de calcul des algorithmes traditionnels, nous avons adopté des techniques de vectorisation et de parallélisation. La vectorisation permet de traiter des opérations de comparaison de manière plus efficace, tandis que la parallélisation exploite les architectures de calcul moderne pour réduire le temps d'exécution.

Analyse critique des résultats obtenus

Les résultats obtenus montrent une amélioration significative par rapport aux méthodes traditionnelles. L'intégration des prédicats temporels d'Allen a permis de capturer des motifs plus complexes et de mieux représenter les relations entre les événements. La vectorisation des opérations de calcul a permis de réduire le temps nécessaire pour analyser les données, et la parallélisation a encore accéléré le processus en utilisant pleinement les capacités des architectures modernes. Les expérimentations ont été menées sur une base de données financière de grande envergure, et les résultats démontrent une amélioration du temps d'exécution et de la précision des motifs extraits. Cependant, des défis subsistent, notamment en termes de gestion de la complexité en espace pour des ensembles de données encore plus volumineux.

Quelques perspectives

Bien que notre étude se soit concentrée sur les bases de données financières, les techniques développées pourraient être appliquées à d'autres domaines où les données temporelles jouent un rôle crucial, comme la santé, l'analyse des réseaux sociaux, et la gestion des chaînes d'approvisionnement. La validation de notre approche dans ces contextes pourrait démontrer sa polyvalence et son utilité généralisée. En outre, ces techniques pourraient également être étendues à la prévision des conditions météorologiques, à la prévision des prix des actions en bourse, à l'évolution du PIB, à la progression des maladies, ainsi qu'à l'analyse du comportement des utilisateurs en ligne. L'application de nos méthodes dans ces domaines pourrait offrir des perspectives précieuses et contribuer de manière significative à des décisions éclairées et à des stratégies optimisées dans diverses industries.

Un autre axe de recherche intéressant serait l'optimisation automatique des paramètres de l'algorithme, tels que les seuils de support et de confiance, pour maximiser la qualité et la quantité des motifs temporels découverts. L'application de techniques d'apprentissage automatique pourrait aider à ajuster ces paramètres de manière dynamique en fonction des caractéristiques spécifiques des données analysées.

La réduction de la complexité en espace constitue une autre direction prometteuse pour améliorer l'efficacité de notre approche. En optimisant la gestion de la mémoire et en réduisant l'empreinte mémoire des algorithmes utilisés, nous pourrions traiter des ensembles de données plus volumineux sans compromettre les performances. Cela inclut l'utilisation de structures de données plus compactes et l'optimisation des processus de stockage et de récupération des informations pertinentes, permettant ainsi de maximiser l'utilisation des ressources disponibles tout en minimisant la consommation de mémoire.

De plus, l'intégration de nouveaux types de faits temporels au-delà de l'augmentation et de la diminution pourrait enrichir l'analyse.

BIBLIOGRAPHIE

[1] Xiaoxiao Kong GC Qiang Wei. An approach to discovering multi-temporal patterns and its application to financial databases. Information Sciences. 2010;180 :873-85.

[2] Collard M. Fouille de données, Contributions Méthodologiques et Applicatives [Habilitation à diriger des recherches]. Université Nice Sophia Antipolis; 2003. Available from : https://theses.hal.science/tel-01059407.

[3] Kobia. Qu'est-ce que le Machine Learning ?; 2023. Accessed : 2024-07-12. https://kobia.fr/quest-ce-que-le-machine-learning/.

[4] Aspexit. Réseau de neurones : On va essayer de démystifier un peu tout ça !; 2023. Accessed : 2024-07-12. https://www.aspexit.com/reseau-de-neurones-on-va-essayer-de-demystifier-un-peu-tout-ca-1/.

[5] Linda S. Support Vector Machine (SVM) Algorithm; 2023. Accessed : 2024-07-12. https://medium.com/@sumbatilinda/support-vector-machine-svm-algorithm-064566b5d411.

[6] Digital B. Metode K-Means Clustering Berbasis TF-IDF Sebagai Dokumen Persyaratan Perangkat Lunak yang Ditulis dalam Bahasa Mandarin; 2023. Accessed : 2024-07-12. https://www.kompasiana.com/bisnisdigital2022d0924/654a6c6fee794a4b591334a2/metode-k-means-clustering-berbasis-tf-idf-sebagai-dokumen-persyaratan-p
page=2&page_images=1#.

[7] Kamga YFI. Predicting RNA Secondary Structure : An Efficient Computation Based on Automatic Parallelization Techniques. Unknown; 2020.

[8] Kengne Tchendji V. Solutions parallèles efficaces sur le modèle CGM d'une classe de problèmes issus de la programmation dynamique [PhD thesis]. Université de Picardie - Jules Verne; Université de Yaoundé I, Cameroun; 2014. Available from : https://hal.science/tel-01450981.

[9] Al-Aubidy KM. Parallel Computer Models ; 2020. Lecture Notes, Advanced Computer Architecture (0630561), Lecture 8. https://www.philadelphia.edu.jo/academics/kaubaidy/uploads/ACA-Lect18.pdf.

[10] Laboratory LLN. Introduction to Parallel Computing Tutorial ; 2020. Available from : https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial.

[11] Batal I, Fradkin D, Harrison J, Moerchen F, Hauskrecht M. Mining Recent Temporal Patterns for Event Detection in Multivariate Time Series Data. KDD : proceedings International Conference on Knowledge Discovery amp ; Data Mining. 2012 ;2012 :280—288. Available from : https://europepmc.org/articles/PMC4414327.

[12] Fayyad UM, Piatetsky-Shapiro G, Smyth P. Knowledge discovery and data mining : Towards a unifying framework. In : Proceedings of the second international conference on Knowledge discovery and data mining (KDD 96) ; 1996. p. 82-8.

[13] Han J, Kamber M, Pei J. Data Mining : Concepts and Techniques. Morgan Kaufmann ; 2011.

[14] Seber GAF, Lee AJ. Linear Regression Analysis. Wiley ; 2012.

[15] Drucker H, Burges CJC, Kaufman L, Smola A, Vapnik V. Support Vector Regression Machines. Advances in Neural Information Processing Systems. 1997 ;9 :155-61.

[16] Hoerl AE, Kennard RW. Ridge Regression : Biased Estimation for Nonorthogonal Problems. Technometrics. 1970 ;12(1) :55-67.

[17] Tibshirani R. Regression Shrinkage and Selection via the Lasso. Journal of the Royal Statistical Society : Series B (Methodological). 1996 ;58(1) :267-88.

[18] Jain AK, Murty MN, Flynn PJ. Data clustering : A review. ACM Computing Surveys (CSUR). 1999 ;31(3) :264-323.

[19] MacQueen J. Some methods for classification and analysis of multivariate observations. In : Proceedings of the fifth Berkeley symposium on mathematical statistics and probability. vol. 1. University of California Press ; 1967. p. 281-97.

[20] Agrawal R, Imieliński T, Swami A. Mining association rules between sets of items in large databases. 1993 :207-16.

[21] Han J, Kamber M, Pei J. Data Mining : Concepts and Techniques. Morgan Kaufmann ; 2006.

[22] Agrawal R, Srikant R. Fast algorithms for mining association rules. In : Proc. 20th int. conf. very large data bases, VLDB. vol. 1215 ; 1994. p. 487-99.

[23] Box GE, Jenkins GM. Time Series Analysis : Forecasting and Control. Holden-Day ; 1976.

[24] Hyndman RJ, Koehler AB, Ord JK, Snyder RD. Forecasting with Exponential Smoothing : The State Space Approach. Springer ; 2008.

[25] Srikant R, Agrawal R. Mining sequential patterns : Generalizations and performance improvements. In : International Conference on Extending Database Technology. Springer ; 1996. p. 3-17.

[26] Pei J, Han J, Mortazavi-Asl B, Wang J, Pinto H, Chen Q, et al. Prefixspan : Mining sequential patterns efficiently by prefix-projected pattern growth. In : Proceedings 17th International Conference on Data Engineering. IEEE ; 2001. p. 215-24.

[27] Kandel S, Paepcke A, Hellerstein J, Heer J. Enterprise data analysis and visualization : An interview study. IEEE Transactions on Visualization and Computer Graphics. 2012 ;18(12) :2917-26.

[28] Mayer-Schönberger V, Cukier K. Big Data : A Revolution That Will Transform How We Live, Work, and Think. Houghton Mifflin Harcourt ; 2013.

[29] Wu X, Kumar V, Quinlan JR, Ghosh J, Yang Q, Motoda H, et al. Top 10 algorithms in data mining. Knowledge and Information Systems. 2008 ;14(1) :1-37.

[30] Ozdamli F, Ozdal H. Life-long Learning Competence Perceptions of the Teachers and Abilities in Using Information-Communication Technologies. Procedia - Social and Behavioral Sciences. 2015 ;182 :718-25.

[31] Feldmann A, Gasser O, Lichtblau F, Pujol E, Poese I, Dietzel C, et al. The Lockdown Effect : Implications of the COVID-19 Pandemic on Internet Traffic. 2020 :1-18.

[32] Pacheco P, Malensek M. An Introduction to Parallel Programming. Morgan Kaufmann ; 2021.

[33] Sterling T, Anderson M, Brodowicz M. High Performance Computing : Modern Systems and Practices. Morgan Kaufmann ; 2017.

[34] Loshin D. Business Intelligence : The Savvy Managers Guide. Morgan Kaufmann ; 2013.

[35] Kengne Tchendji V. Solutions parallèles efficaces sur le modèle CGM d'une classe de problèmes issus de la programmation dynamique [PhD thesis]. Université de Picardie Jules Verne ; 2014.

[36] Steffen M, Divincenzo DP, Chow JM, Theis TN, Ketchen MB. Quantum computing : An IBM perspective. IBM Journal of Research and Development. 2011 ;55(5) :1-13.

[37] Flynn MJ. Some computer organizations and their effectiveness. IEEE Transactions on Computers. 1972 ;C-21(9) :948-60.

[38] Raina SK. Virtual shared memory : A survey of techniques and systems. University of Bristol, Bristol, UK ; 1992.

[39] Wasserman S. Parallel Computing : Architectures, Algorithms and Applications. Morgan Kaufmann Publishers ; 1994.

[40] Kengne V. Solutions paralleles efficaces sur le modele CGM d'une classe de problemes issus de la programmation dynamique. Université de Picardie Jules Verne (France), Université de Yaoundé 1 (Cameroun) ; 2014.

[41] Diaz J, Munoz-Caro C, Nino A. A survey of parallel programming models and tools in the multi and many-core era. IEEE Transactions on parallel and distributed systems. 2012 ;23(8) :1369-86.

[42] Dalcin L. Mpi for python ; 2020. Available from : https://bitbucket.org/mpi4py/mpi4py.

[43] Kang SJ, Lee SY, Lee KM. Performance comparison of openmp, mpi, and mapreduce in practical problems. Advances in Multimedia 2015. 2015 :575687 :1-575687 :9.

[44] Winarko E, Roddick JF. ARMADA : An algorithm for discovering richer relative temporal association rules from interval-based data. Data & Knowledge Engineering. 2007 ;63(1) :76-90.

[45] Zhang L, Chen G, Brijs T, Zhang X. Discovering during-temporal patterns (DTPs) in large temporal databases. Proceedings of the 2008 IEEE International Conference on Data Mining. 2008 ;8(12) :929-34.

[46] Allen JF. Maintaining knowledge about temporal intervals. Communications of the ACM. 1983 ;26(11) :832-43.

[47] Shanghai Wind Information Co , Ltd . Wind Financial Databases ; 2021. Available from : http://www.wind.com.cn/.